

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1



BÁO CÁO
CÁC HỆ THỐNG PHÂN TÁN

ĐỀ TÀI:

**ỨNG DỤNG KIẾN TRÚC MICROSERVICES TRONG XÂY
DỰNG HỆ THỐNG ĐẶT VÉ SỰ KIỆN FLASH TICKET**

Giảng viên hướng dẫn	Kim Ngọc Bách
Lớp	D22HTTT3
Nhóm	10
Thành viên	MSV
Lê Văn Minh	B22DCCN533
Phạm Văn Tuyền	B22DCCN773

Hà Nội, 2026

LỜI CẢM ƠN

Trong suốt quá trình học tập và thực hiện bài tập lớn này, nhóm em đã nhận được sự hỗ trợ và tạo điều kiện rất lớn từ phía nhà trường cùng các thầy.

Nhóm em xin gửi lời tri ân sâu sắc nhất đến thầy Kim Ngọc Bách. Thầy đã trực tiếp, hướng dẫn, định hướng và truyền đạt những kiến thức, kinh nghiệm thực tiễn quý báu để nhóm em có thể tháo gỡ khó khăn và hoàn thành tốt đề tài nghiên cứu này.

Mặc dù đã nỗ lực hết mình trong quá trình nghiên cứu và phát triển hệ thống, dự án chắc chắn vẫn khó tránh khỏi những thiếu sót nhất định về mặt kỹ thuật lẫn lý luận. Nhóm em rất mong nhận được sự quan tâm và những lời góp ý quý báu từ thầy để đề tài được hoàn thiện hơn trong tương lai.

Em xin chân thành cảm ơn!

MỤC LỤC

LỜI CẢM ƠN.....	1
DANH MỤC HÌNH ẢNH.....	6
DANH MỤC BẢNG BIỂU.....	7
DANH MỤC TỪ VIẾT TẮT.....	9
PHÂN CHIA CÔNG VIỆC.....	10
CHƯƠNG 1: TỔNG QUAN VỀ HỆ THỐNG.....	11
1.1. Tổng quan về hệ thống đặt vé sự kiện trực tuyến.....	11
1.2. Bối cảnh và lý do chọn đề tài.....	12
1.3. Mục tiêu của đề tài.....	12
1.4. Đối tượng sử dụng hệ thống.....	13
1.4.1. Người mua vé.....	14
1.4.2. Nhà tổ chức sự kiện.....	14
1.4.3. Quản trị viên hệ thống.....	15
1.5. Phạm vi chức năng của hệ thống.....	15
1.6. Đóng góp của đề tài.....	16
1.7. Giới hạn của đề tài.....	17
1.8. Kết chương.....	17
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG.....	18
2.1. Kiến trúc Microservice.....	18
2.1.1. Khái niệm Microservice.....	18
2.1.2. Đặc điểm của kiến trúc Microservice.....	18
2.1.3. Ưu điểm và hạn chế của microservice.....	19
2.2. Giao tiếp đồng bộ bằng RESTful API.....	20
2.3. Giao tiếp bất đồng bộ bằng Message Queue.....	20
2.3.1. Khái niệm Message Queue.....	21
2.3.2. Khái niệm RabbitMQ.....	21
2.4. API Gateway và Service Discovery.....	22
2.4.1. API Gateway.....	22
2.4.2. Eureka Service Discovery.....	22
2.4.3. Spring Cloud Config Server.....	23
2.5. Xác thực và phân quyền bằng Keycloak, OAuth2 và JWT.....	23
2.6. Cơ sở dữ liệu trong hệ Microservice.....	24
2.6.1. PostgreSQL cho dữ liệu nghiệp vụ.....	24
2.6.2. MongoDB cho dữ liệu người dùng.....	25
2.6.3. Redis cho distributed lock và chống xử lý trùng.....	26

2.6.4. PGVector cho tìm kiếm ngữ nghĩa.....	26
2.7. Một số kỹ thuật nâng cao trong hệ thống.....	27
2.7.1. Circuit breaker và retry policy.....	27
2.7.2. Distributed locking trong đặt vé.....	27
2.7.3. Event-driven workflow.....	28
2.7.4. RAG và chatbot hỗ trợ tìm kiếm sự kiện.....	29
2.8. Kết chương.....	30
CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG FLASH TICKET.....	31
3.1. Phân tích yêu cầu hệ thống.....	31
3.1.1. Yêu cầu chức năng chính.....	31
3.1.2. Yêu cầu kiến trúc Microservice.....	31
3.1.3. Tác nhân sử dụng hệ thống.....	32
3.2. Thiết kế kiến trúc tổng thể hệ thống.....	32
3.2.1. Kiến trúc Microservice của Flash Ticket.....	33
3.2.2. Phân tách trách nhiệm giữa các service.....	33
3.2.3. Giao tiếp đồng bộ bằng RESTful API.....	34
3.2.4. Giao tiếp bất đồng bộ bằng RabbitMQ.....	35
3.3. Thiết kế các Microservice trong hệ thống.....	36
3.3.1. API Gateway.....	36
3.3.2. Core Service.....	37
3.3.3. User Service.....	38
3.3.4. Discovery Service.....	39
3.3.5. Eureka Server và Config Server.....	40
3.4. Thiết kế API Gateway.....	41
3.4.1. Vai trò của API Gateway.....	42
3.4.2. Định tuyến request đến các microservice.....	43
3.4.3. Load-balanced route thông qua Eureka.....	43
3.4.4. Xác thực và phân quyền tại Gateway.....	44
3.4.5. Rate limiting bằng Redis.....	44
3.4.6. Fallback khi service không phản hồi.....	45
3.5. Thiết kế khả năng chịu lỗi với Resilience4j.....	46
3.5.1. Circuit breaker trong API Gateway.....	46
3.5.2. Circuit breaker cho Core Service, User Service và Discovery Service...47	
3.5.3. Retry policy cho request GET.....	48
3.5.4. Time limiter cho các service.....	49
3.5.5. Ý nghĩa của cơ chế chịu lỗi trong hệ thống đặt vé.....	50
3.6. Thiết kế Service Discovery bằng Eureka.....	50
3.6.1. Vai trò của Eureka Server.....	51

3.6.2. Cơ chế service đăng ký vào Eureka.....	51
3.6.3. Cơ chế heartbeat và theo dõi trạng thái service.....	52
3.6.4. Lợi ích của Eureka khi mở rộng hệ thống.....	53
3.7. Thiết kế quản lý cấu hình tập trung bằng Spring Cloud Config Server.....	53
3.8. Thiết kế event-driven workflow bằng RabbitMQ.....	54
3.8.1. Vai trò của RabbitMQ trong hệ thống.....	55
3.8.2. Luồng xử lý sau khi thanh toán thành công.....	55
3.8.3. Luồng phát hành vé, tạo QR và gửi email xác nhận.....	56
3.8.4. Luồng đồng bộ sự kiện sang Discovery Service.....	57
3.8.5. Luồng đồng bộ người dùng từ Keycloak sang User Service.....	58
3.8.6. Manual ACK, NACK và Dead Letter Queue.....	59
3.8.8. Publish event sau khi transaction commit.....	60
3.9. Thiết kế Redis trong hệ thống.....	61
3.9.1. Vai trò của Redis và Redisson.....	61
3.9.2. Redis rate limiting tại API Gateway.....	62
3.9.3. Redis spam lock khi tạo booking.....	62
3.9.4. Redis distributed lock theo ticket type và seat.....	63
3.9.5. Redis spam lock khi tạo URL thanh toán.....	63
3.9.6. Redis idempotency lock khi xử lý VNPay IPN.....	64
3.9.7. Redis cache trạng thái ghế trong seat map.....	65
3.9.8. Kết hợp Redis lock, double-check stock và atomic SQL update.....	65
3.10. Thiết kế luồng nghiệp vụ trọng tâm.....	66
3.10.1. Luồng đăng nhập và phân quyền.....	67
3.10.2. Luồng tạo và quản lý sự kiện.....	67
3.10.3. Luồng thiết kế sơ đồ ghế.....	68
3.10.4. Luồng đặt vé và giữ vé.....	69
3.10.5. Luồng thanh toán qua VNPay.....	69
3.10.6. Luồng xử lý VNPay IPN.....	70
3.10.7. Luồng phát hành vé QR và gửi email.....	70
3.10.8. Luồng check-in vé tại sự kiện.....	70
3.10.9. Luồng hoàn vé/ghế khi đơn hàng hết hạn.....	71
3.11. Thiết kế cơ sở dữ liệu.....	71
3.12.1. Cơ sở dữ liệu Core Service.....	72
3.12.2. Cơ sở dữ liệu User Service.....	73
3.12.3. Cơ sở dữ liệu Discovery Service và PGVector.....	73
3.12.4. Quan hệ dữ liệu trong nghiệp vụ đặt vé.....	74
3.12.5. Dữ liệu seat map, ticket type và seat inventory.....	75
3.13. Triển khai hệ thống trên AWS EC2 và Nginx.....	76

3.14. Kết luận chương.....	77
CHƯƠNG 4: XÂY DỰNG VÀ TRIỂN KHAI HỆ THỐNG.....	78
4.1. Môi trường phát triển và công nghệ sử dụng.....	78
4.1.1. Backend.....	79
4.1.2. Frontend.....	79
4.1.3. Cơ sở dữ liệu và hạ tầng.....	79
4.1.4. Công cụ triển khai.....	79
4.3. Giao diện hệ thống.....	80
4.3.1. Giao diện trang chủ và tìm kiếm sự kiện.....	80
4.3.2. Giao diện chi tiết sự kiện.....	81
4.3.3. Giao diện chọn vé và thanh toán.....	81
4.3.4. Giao diện vé của tôi và đơn hàng của tôi.....	82
4.3.5. Giao diện chatbot hỗ trợ người dùng.....	84
4.4. Kết chương.....	85
CHƯƠNG 5: ĐÁNH GIÁ VÀ KẾT LUẬN.....	86
5.1. Ưu điểm của hệ thống.....	86
5.2. Hạn chế còn tồn tại.....	86
5.3. Hướng phát triển trong tương lai.....	87
5.4. Kết luận.....	87
LINK GITHUB.....	88

DANH MỤC HÌNH ẢNH

Hình 1. Quy trình đặt vé tổng quát. Nguồn: 360-ticketing.com.....	11
Hình 2. Giới thiệu kiến trúc Microservices. Nguồn: akamai.com.....	18
Hình 3. Giới thiệu RESTful API. Nguồn: vnpro.vn.....	20
Hình 4. Giới thiệu Message Queue. Nguồn: viblo.asia.....	21
Hình 6. Giới thiệu API Gateway. Nguồn: blog.algomaster.io.....	22
Hình 7. Giới thiệu Eureka. Nguồn: dineshonjava.com.....	22
Hình 7. Spring Cloud Config Server.....	23
Hình 8. Quy trình tích hợp Keycloak vào hệ thống ký số. Nguồn: viblo.asia.....	23
Hình 9. Giới thiệu PostgreSQL. Nguồn: viblo.asia.....	24
Hình 10. Giới thiệu MongoDB. Nguồn: viblo.asia.....	25
Hình 11. Giới thiệu Redis cho distributed lock. Nguồn: viblo.asia.....	26
Hình 12. Giới thiệu Circuit breaker và retry policy. Nguồn: dev.to.....	27
Hình 13. Giới thiệu Distributed locking. Nguồn: anhdh.net.....	27
Hình 14. Giới thiệu Event-driven. Nguồn: linkedin.com.....	28
Hình 15. Giới thiệu Retrieval-Augmented Generation. Nguồn: medium.com.....	29
Hình 16. Kiến trúc tổng thể hệ thống Flash Ticket.....	33
Hình 17. Các luồng bất đồng bộ chính trong hệ thống Flash Ticket.....	36
Hình 18. API Gateway trong hệ thống Flash Ticket.....	41
Hình 19. Luồng xử lý request qua API Gateway.....	43
Hình 20. Trạng thái circuit breaker trong API Gateway.....	47
Hình 21. Retry policy cho request GET đến Core Service.....	49
Hình 22. Cơ chế service đăng ký vào Eureka.....	52
Hình 23. Luồng phát hành vé, tạo QR và gửi email xác nhận.....	57
Hình 24. Kết hợp Redis lock và database trong luồng booking.....	66
Hình 25. Luồng đăng nhập và phân quyền bằng Keycloak, OAuth2 và JWT.....	67
Hình 26. Giao diện trang chủ.....	80
Hình 27. Giao diện trang tìm kiếm.....	81
Hình 28. Giao diện trang chi tiết sự kiện.....	81
Hình 29. Giao diện trang danh sách vé.....	82
Hình 30. Giao diện trang chi tiết vé.....	83
Hình 31. Giao diện trang danh sách đơn hàng.....	83
Hình 32. Giao diện trang chi tiết đơn hàng.....	84
Hình 33. Giao diện Chatbot.....	85

DANH MỤC BẢNG BIỂU

Bảng 1: So sánh đặt vé truyền thống và đặt vé trực tuyến.....	12
Bảng 2. Mục tiêu của đề tài.....	13
Bảng 3. Các nhóm người dùng chính của hệ thống.....	13
Bảng 4. Phạm vi chức năng của hệ thống Flash Ticket.....	16
Bảng 5. Đặc điểm của kiến trúc microservice trong Flash Ticket.....	19
Bảng 6. Các schema PostgreSQL chính trong hệ thống.....	24
Bảng 7. Các collection MongoDB chính của User Service.....	25
Bảng 8. Các lớp bảo vệ trong luồng đặt vé.....	28
Bảng 9. Yêu cầu chức năng chính của hệ thống Flash Ticket.....	31
Bảng 10. Phân tách trách nhiệm giữa các service.....	32
Bảng 11. Tác nhân sử dụng và tương tác với hệ thống Flash Ticket.....	32
Bảng 12. Phân tách trách nhiệm giữa các service.....	34
Bảng 13. Thiết kế định tuyến RESTful API qua API Gateway.....	35
Bảng 14. Các nhóm route chính của API Gateway.....	37
Bảng 15. Các nhóm chức năng chính của Core Service.....	38
Bảng 16. Các nhóm chức năng chính của User Service.....	39
Bảng 17. Các nhóm chức năng chính của Discovery Service.....	39
Bảng 18. Vai trò của Eureka Server và Config Server.....	40
Bảng 19. Vai trò của API Gateway trong hệ thống Flash Ticket.....	42
Bảng 20. Cấu hình RedisRateLimiter tại Gateway.....	44
Bảng 21. Fallback được cấu hình tại API Gateway.....	45
Bảng 22. Circuit breaker được cấu hình tại API Gateway.....	46
Bảng 23. Cấu hình circuit breaker trong Gateway.....	47
Bảng 24. Cấu hình time limiter cho các service.....	49
Bảng 25. Ý nghĩa của Resilience4j trong Flash Ticket.....	50
Bảng 26. Vai trò của Eureka Server trong hệ thống.....	51
Bảng 27. Ý nghĩa của heartbeat trong Eureka.....	52
Bảng 28. Lợi ích của Eureka trong Flash Ticket.....	53
Bảng 29. Vai trò của Config Server trong Flash Ticket.....	54
Bảng 30. Vai trò của RabbitMQ trong hệ thống Flash Ticket.....	55
Bảng 31. Các bước xử lý sau khi VNPay IPN thành công.....	56
Bảng 32. Luồng đồng bộ sự kiện sang Discovery Service.....	58
Bảng 33. Đồng bộ người dùng từ Keycloak sang User Service.....	59
Bảng 34. Cách xử lý ACK/NACK trong các listener.....	59
Bảng 35. Các event được publish sau transaction commit.....	60
Bảng 36. Vai trò của Redis và Redisson trong hệ thống.....	61
Bảng 37. Cấu hình rate limiting tại API Gateway.....	62

Bảng 38. Redis spam lock khi tạo booking.....	62
Bảng 39. Distributed lock trong luồng booking.....	63
Bảng 40. Redis spam lock khi tạo URL thanh toán.....	64
Bảng 41. Redis idempotency lock trong VNPay IPN.....	64
Bảng 42. Redis cache trạng thái ghế.....	65
Bảng 43. Các thao tác quản lý sự kiện của organizer.....	68
Bảng 44. Các bước xử lý khi publish seat map.....	68
Bảng 45. Các bước khởi tạo thanh toán VNPay.....	69
Bảng 46. Các bước xử lý check-in vé.....	71
Bảng 47. Các bước hoàn vé/ghế khi order hết hạn.....	71
Bảng 48. Các schema chính trong Core Service.....	72
Bảng 49. Các collection chính của User Service.....	73
Bảng 50. Dữ liệu chính trong Discovery Service.....	74
Bảng 51. Vai trò các bảng trong luồng đặt vé.....	74
Bảng 52. Các bảng liên quan đến seat map.....	75
Bảng 53. Công nghệ sử dụng trong hệ thống Flash Ticket.....	78

DANH MỤC TỪ VIẾT TẮT

STT	Từ viết tắt	Ý nghĩa
1	API	Application Programming Interface
2	REST	Representational State Transfer
3	JWT	JSON Web Token
4	OAuth2	Open Authorization 2.0
5	MQ	Message Queue
6	DLQ	Dead Letter Queue
7	RAG	Retrieval-Augmented Generation
8	LLM	Large Language Model
9	QR	Quick Response
10	EC2	Elastic Compute Cloud

PHÂN CHIA CÔNG VIỆC

Mã sinh viên	Họ và tên	Công việc
B22DCCN533	Lê Văn Minh	Backend Dev
B22DCCN773	Phạm Văn Tuyển	Frontend Dev

CHƯƠNG 1: TỔNG QUAN VỀ HỆ THỐNG

1.1. Tổng quan về hệ thống đặt vé sự kiện trực tuyến

Trong những năm gần đây, nhu cầu tham gia các sự kiện giải trí, âm nhạc, hội thảo, triển lãm và các chương trình cộng đồng ngày càng tăng. Cùng với sự phát triển của thương mại điện tử và thanh toán số, hình thức đặt vé trực tuyến đã dần thay thế cách mua vé truyền thống tại quầy hoặc thông qua trung gian thủ công.

Hệ thống đặt vé sự kiện trực tuyến là nền tảng cho phép người dùng tìm kiếm sự kiện, xem thông tin chi tiết, chọn loại vé, đặt vé, thanh toán và nhận vé điện tử thông qua Internet. Vé sau khi thanh toán thành công thường được gửi dưới dạng mã QR hoặc mã định danh điện tử, giúp quá trình kiểm tra vé tại cổng sự kiện diễn ra nhanh chóng và chính xác hơn.



Hình 1. Quy trình đặt vé tổng quát. Nguồn: 360-ticketing.com

So với phương thức bán vé truyền thống, hệ thống đặt vé trực tuyến có nhiều ưu điểm như giảm thời gian chờ đợi, hạn chế sai sót trong quá trình bán vé, hỗ trợ thống kê doanh thu theo thời gian thực và giúp nhà tổ chức quản lý sự kiện hiệu quả hơn.

Bảng 1: So sánh đặt vé truyền thống và đặt vé trực tuyến

Tiêu chí	Đặt vé truyền thống	Đặt vé trực tuyến
Cách mua vé	Mua trực tiếp tại quầy hoặc qua trung gian	Mua qua website hoặc ứng dụng
Thanh toán	Tiền mặt hoặc chuyển khoản thủ công	Thanh toán trực tuyến qua cổng thanh toán
Nhận vé	Vé giấy	Vé điện tử, QR Code
Quản lý tồn vé	Dễ sai sót, khó cập nhật tức thời	Cập nhật tự động theo thời gian thực
Thống kê	Thực hiện thủ công	Có thể tổng hợp tự động
Kiểm tra vé	Kiểm tra bằng mắt hoặc danh sách	Quét QR Code, xác thực hệ thống

1.2. Bối cảnh và lý do chọn đề tài

Trong thực tế, các hệ thống bán vé sự kiện thường phải xử lý nhiều bài toán phức tạp. Một sự kiện có thể có nhiều loại vé, số lượng vé giới hạn, nhiều người mua cùng lúc, yêu cầu thanh toán nhanh và cần đảm bảo không xảy ra tình trạng bán vượt số lượng vé. Bên cạnh đó, nhà tổ chức cần có công cụ để tạo sự kiện, quản lý vé, theo dõi doanh thu và kiểm tra vé khi người tham dự đến sự kiện.

Vì vậy, đề tài “**Ứng dụng kiến trúc Microservice trong hệ thống đặt vé sự kiện Flash Ticket**” lựa chọn xây dựng hệ thống đặt vé sự kiện theo kiến trúc microservice. Mỗi service đảm nhiệm một nhóm chức năng riêng, có thể phát triển và triển khai độc lập.

1.3. Mục tiêu của đề tài

Mục tiêu chính của đề tài là xây dựng một hệ thống đặt vé sự kiện trực tuyến có khả năng vận hành theo kiến trúc microservice, trong đó các service được tách theo miền chức năng và phối hợp với nhau thông qua RESTful API và message queue.

Bảng 2. Mục tiêu của đề tài

Mục tiêu	Nội dung
Xây dựng hệ thống đặt vé trực tuyến	Cho phép người dùng tìm kiếm sự kiện, chọn vé, đặt vé và thanh toán
Áp dụng kiến trúc microservice	Tách hệ thống thành nhiều service như Core Service, User Service, Discovery Service
Thiết kế RESTful API	Cung cấp API rõ ràng cho frontend và giao tiếp giữa các service
Tích hợp message queue	Sử dụng RabbitMQ cho các luồng bất đồng bộ như đồng bộ user, phát hành vé, gửi email
Đảm bảo xác thực và phân quyền	Dùng Keycloak, OAuth2 và JWT để quản lý đăng nhập, vai trò người dùng
Xử lý đặt vé an toàn	Dùng Redis lock và kiểm tra tồn vé để hạn chế đặt trùng, oversell
Tích hợp thanh toán	Sử dụng VNPay sandbox cho luồng thanh toán trực tuyến
Hỗ trợ AI chatbot	Tích hợp Discovery Service để hỗ trợ tìm kiếm sự kiện và tư vấn đặt vé

1.4. Đối tượng sử dụng hệ thống

Hệ thống Flash Ticket được thiết kế cho ba nhóm đối tượng chính: người mua vé, nhà tổ chức sự kiện và quản trị viên hệ thống. Mỗi nhóm người dùng có vai trò, nhu cầu và quyền truy cập khác nhau.

Bảng 3. Các nhóm người dùng chính của hệ thống

Đối tượng	Vai trò chính	Nhu cầu sử dụng
Người mua vé	Tìm kiếm, đặt vé, thanh toán, nhận vé	Mua vé nhanh, xem lịch sử đơn hàng, nhận QR Code

Nhà tổ chức sự kiện	Tạo và quản lý sự kiện	Quản lý thông tin sự kiện, loại vé, hình ảnh, check-in
Quản trị viên	Quản lý và kiểm duyệt hệ thống	Duyệt organizer, quản lý trạng thái tài khoản, giám sát hệ thống

1.4.1. Người mua vé

Người mua vé là nhóm người dùng phổ biến nhất của hệ thống. Họ có thể truy cập trang chủ, tìm kiếm sự kiện theo từ khóa, danh mục hoặc địa điểm, xem chi tiết sự kiện, chọn loại vé và tiến hành đặt vé.

Sau khi đặt vé, hệ thống tạo đơn hàng ở trạng thái chờ thanh toán. Người dùng có thể thanh toán thông qua cổng VNPay. Khi thanh toán thành công, hệ thống phát hành vé điện tử và tạo mã QR để người dùng sử dụng khi tham gia sự kiện.

Các chức năng chính của người mua vé gồm:

- Đăng nhập bằng tài khoản Keycloak.
- Xem danh sách sự kiện.
- Tìm kiếm và xem chi tiết sự kiện.
- Chọn vé và tạo đơn hàng.
- Thanh toán trực tuyến.
- Xem đơn hàng đã đặt.
- Xem vé điện tử và QR Code.
- Sử dụng chatbot để được hỗ trợ tìm kiếm sự kiện.

1.4.2. Nhà tổ chức sự kiện

Nhà tổ chức sự kiện là người hoặc đơn vị có nhu cầu đăng tải, quản lý và bán vé cho sự kiện của mình. Trong hệ thống, người dùng thông thường có thể đăng ký trở thành organizer. Sau khi được quản trị viên duyệt, tài khoản sẽ được cấp quyền organizer. Nhà tổ chức có thể tạo sự kiện mới, cập nhật thông tin sự kiện, quản lý hình ảnh, tạo loại vé, thiết lập số lượng vé và công bố sự kiện để người mua có thể nhìn thấy.

Các chức năng chính của nhà tổ chức gồm:

- Đăng ký trở thành organizer.
- Quản lý hồ sơ organizer.
- Tạo và chỉnh sửa sự kiện.
- Quản lý hình ảnh sự kiện.
- Quản lý loại vé.
- Công bố hoặc hủy sự kiện.
- Check-in vé bằng QR Code.
- Theo dõi thông tin cơ bản về sự kiện và vé đã bán.

1.4.3. Quản trị viên hệ thống

Quản trị viên là nhóm người dùng có quyền cao nhất trong hệ thống. Vai trò chính của quản trị viên là đảm bảo hệ thống hoạt động ổn định, kiểm duyệt các yêu cầu đăng ký organizer và hỗ trợ quản lý người dùng.

Trong phạm vi đề tài, chức năng quản trị tập trung vào việc duyệt hồ sơ organizer. Khi một người dùng gửi yêu cầu trở thành nhà tổ chức, quản trị viên có thể phê duyệt hoặc từ chối. Nếu được phê duyệt, hệ thống sẽ cập nhật trạng thái organizer trong cơ sở dữ liệu và gán role tương ứng trong Keycloak.

Các chức năng chính của quản trị viên gồm:

- Xem danh sách hồ sơ organizer đang chờ duyệt.
- Phê duyệt hoặc từ chối hồ sơ organizer.
- Gán quyền organizer cho người dùng.
- Theo dõi trạng thái hoạt động cơ bản của hệ thống.

1.5. Phạm vi chức năng của hệ thống

Hệ thống Flash Ticket tập trung vào các chức năng chính phục vụ quy trình đặt vé sự kiện trực tuyến. Phạm vi chức năng được chia theo các nhóm nghiệp vụ như sau:

Bảng 4. Phạm vi chức năng của hệ thống Flash Ticket

Nhóm chức năng	Mô tả
Quản lý người dùng	Đồng bộ user từ Keycloak, xem và cập nhật hồ sơ
Quản lý organizer	Đăng ký, duyệt hồ sơ, cập nhật logo/banner
Quản lý sự kiện	Tạo, sửa, công bố, hủy sự kiện
Quản lý vé	Tạo loại vé, quản lý số lượng vé
Đặt vé	Tạo đơn hàng, giữ vé, chống đặt trùng
Thanh toán	Tạo link VNPay, xử lý IPN, cập nhật trạng thái đơn
Phát hành vé	Tạo vé sau thanh toán, sinh QR Code
Gửi email	Gửi email xác nhận vé qua RabbitMQ
Chatbot hỗ trợ	Tìm kiếm sự kiện và hỗ trợ đặt vé bằng AI
Seat map nâng cao	Sơ đồ ghế và chọn ghế cụ thể

1.6. Đóng góp của đề tài

Đề tài Flash Ticket System có một số đóng góp chính như sau:

- **Thứ nhất**, đề tài xây dựng được một hệ thống đặt vé sự kiện có đầy đủ các luồng nghiệp vụ cốt lõi, từ tìm kiếm sự kiện, đặt vé, thanh toán, phát hành vé điện tử đến check-in bằng QR Code.
- **Thứ hai**, hệ thống áp dụng kiến trúc microservice với nhiều service độc lập. Các service được phân chia theo miền chức năng rõ ràng. Đồng thời sử dụng kết hợp giao tiếp đồng bộ và bất đồng bộ. RESTful API được dùng cho các request trực tiếp từ frontend hoặc giữa các service.
- **Thứ ba**, hệ thống tích hợp các công nghệ thực tế như Keycloak cho xác thực, VNPay cho thanh toán, Cloudinary cho lưu trữ ảnh, Redis cho distributed lock.

1.7. Giới hạn của đề tài

Mặc dù hệ thống đã triển khai nhiều chức năng quan trọng, đề tài vẫn tồn tại một số giới hạn nhất định như:

- **Thứ nhất**, hệ thống chủ yếu phục vụ mục tiêu học tập và mô phỏng nghiệp vụ. Một số cấu hình như thanh toán VNPay sandbox, môi trường Docker Compose cục bộ và dữ liệu mẫu chưa thể xem là sẵn sàng cho triển khai thương mại thực tế.
- **Thứ hai**, hệ thống chưa các chức năng nâng cao như hoàn tiền, chuyển nhượng vé, giám sát tập trung từng services chưa được triển khai hoàn chỉnh.
- **Thứ ba**, hệ thống đã có cơ chế xử lý lỗi cơ bản như retry, circuit breaker ở gateway, dead letter queue và logging. Tuy nhiên, các cơ chế production-level như centralized logging, distributed tracing đầy đủ, alerting và monitoring dashboard vẫn cần được phát triển thêm.

1.8. Kết chương

Chương 1 đã trình bày tổng quan về hệ thống đặt vé sự kiện trực tuyến, bối cảnh lựa chọn đề tài, mục tiêu xây dựng hệ thống Flash Ticket, các nhóm người dùng chính, phạm vi chức năng, đóng góp và giới hạn của đề tài.

Flash Ticket System được định hướng là một hệ thống đặt vé sự kiện trực tuyến áp dụng kiến trúc microservice. Đây là nền tảng phù hợp để nghiên cứu, triển khai và đánh giá các kỹ thuật quan trọng trong phát triển ứng dụng phân tán hiện đại.

Các nội dung lý thuyết liên quan đến kiến trúc microservice, RESTful API, message queue, API Gateway, service discovery, xác thực JWT, cơ sở dữ liệu và các kỹ thuật hỗ trợ hệ thống sẽ được trình bày trong Chương 2.

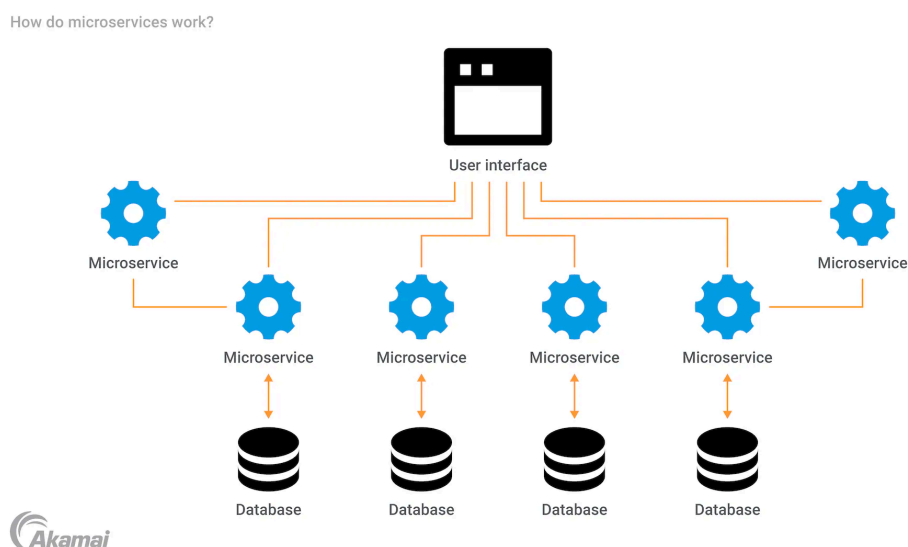
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

2.1. Kiến trúc Microservice

2.1.1. Khái niệm Microservice

Microservice là kiến trúc phần mềm trong đó một hệ thống lớn được chia thành nhiều dịch vụ nhỏ, độc lập với nhau. Mỗi dịch vụ phụ trách một miền nghiệp vụ riêng, có thể được phát triển, triển khai, mở rộng và bảo trì độc lập.

Trong hệ thống Flash Ticket, kiến trúc microservice được áp dụng bằng cách tách hệ thống thành nhiều thành phần như: API Gateway, Core Service, User Service, Discovery Service, Config Server và Eureka Server. Mỗi service đảm nhận một vai trò riêng thay vì gom toàn bộ chức năng vào một ứng dụng nguyên khối.



Hình 2. Giới thiệu kiến trúc Microservices. Nguồn: akamai.com

2.1.2. Đặc điểm của kiến trúc Microservice

Kiến trúc microservice có một số đặc điểm quan trọng sau:

Bảng 5. Đặc điểm của kiến trúc microservice trong Flash Ticket

Đặc điểm	Mô tả	Flash Ticket
Tách biệt nghiệp vụ	Mỗi service phụ trách một miền chức năng riêng	Core Service, User Service, Discovery Service
Triển khai độc lập	Service có thể chạy và cập nhật riêng	Mỗi service Spring Boot có cấu hình và port riêng
Giao tiếp qua API	Các service giao tiếp bằng RESTful API hoặc message queue	Gateway route API, RabbitMQ truyền event
Dữ liệu tách biệt	Mỗi service sở hữu dữ liệu riêng hoặc vùng dữ liệu riêng	PostgreSQL, MongoDB, PGVector
Dễ mở rộng	Có thể scale service có tải cao	Booking/payment có thể scale riêng Core Service
Chịu lỗi tốt hơn	Lỗi ở một service không nhất thiết làm sập toàn hệ thống	Circuit breaker, fallback, queue bất đồng bộ

2.1.3. Ưu điểm và hạn chế của microservice

Kiến trúc microservice phù hợp với các hệ thống có nhiều miền nghiệp vụ, nhiều luồng xử lý và yêu cầu mở rộng linh hoạt. Đối với hệ thống đặt vé, microservice giúp tách riêng các phần quan trọng như người dùng, sự kiện, đặt vé, thanh toán, thông báo và chatbot.

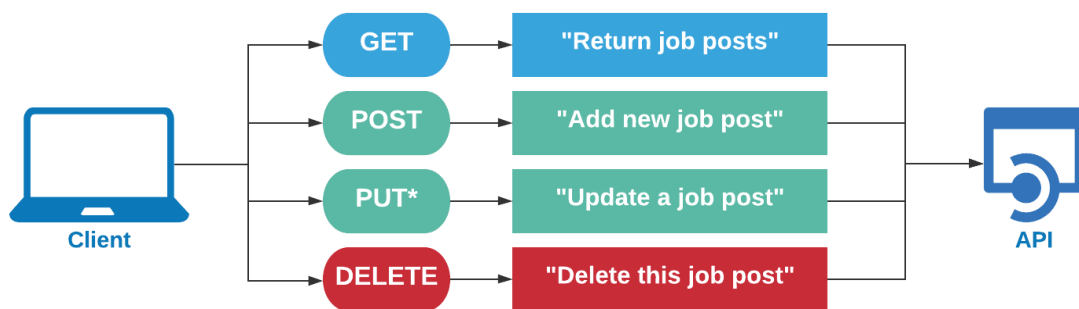
Ưu điểm chính gồm:

- Dễ mở rộng từng service theo nhu cầu tải.
- Dễ bảo trì vì mỗi service có phạm vi chức năng rõ ràng.
- Có thể phát triển song song giữa các nhóm.
- Giảm ảnh hưởng dây chuyền khi một chức năng gặp lỗi.
- Phù hợp với giao tiếp bất đồng bộ bằng message queue.

Tuy nhiên, microservice cũng có một số hạn chế:

- Cấu hình và triển khai phức tạp hơn monolithic.
- Cần xử lý giao tiếp liên service.
- Cần quan tâm đến độ trễ mạng, lỗi mạng và tính nhất quán dữ liệu.
- Debug khó hơn vì một luồng nghiệp vụ có thể đi qua nhiều service.
- Cần cơ chế logging, tracing và monitoring tốt hơn.

2.2. Giao tiếp đồng bộ bằng RESTful API



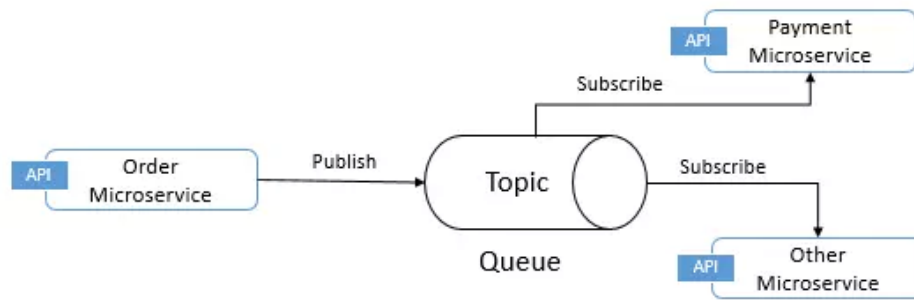
Hình 3. Giới thiệu RESTful API. Nguồn: vnpro.vn

RESTful API là phương thức giao tiếp phổ biến trong các hệ thống web và microservice. REST sử dụng các phương thức HTTP như GET, POST, PUT, PATCH, DELETE để thao tác với tài nguyên.

RESTful API phù hợp với các thao tác cần phản hồi ngay, chẳng hạn như lấy danh sách sự kiện, xem chi tiết sự kiện, tạo đơn hàng hoặc khởi tạo thanh toán. Tuy nhiên, với các tác vụ tốn thời gian như gửi email, phát hành vé hoặc đồng bộ dữ liệu, hệ thống sử dụng message queue để xử lý bất đồng bộ.

2.3. Giao tiếp bất đồng bộ bằng Message Queue

2.3.1. Khái niệm Message Queue



Hình 4. Giới thiệu Message Queue. Nguồn: viblo.asia

Message queue là cơ chế giao tiếp bất đồng bộ giữa các thành phần trong hệ thống. Thay vì service A gọi trực tiếp service B và chờ phản hồi, service A có thể gửi một message vào hàng đợi. Service B sẽ nhận message và xử lý khi sẵn sàng. Cách tiếp cận này giúp giảm phụ thuộc trực tiếp giữa các service. Nếu service nhận tạm thời bị lỗi, message vẫn có thể nằm trong queue để xử lý sau.

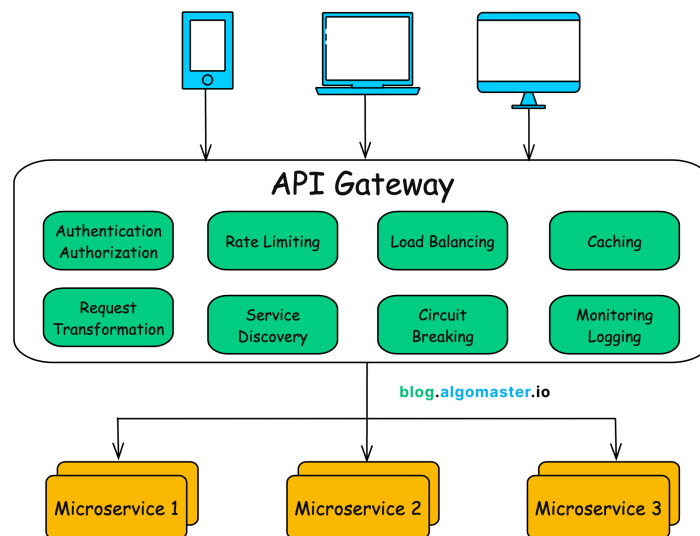
2.3.2. Khái niệm RabbitMQ

RabbitMQ là message broker được sử dụng để truyền message giữa các service. RabbitMQ hoạt động dựa trên các thành phần chính như exchange, queue, binding và routing key.

Trong hệ thống Flash Ticket, RabbitMQ giúp xử lý các công việc hậu kỳ sau khi nghiệp vụ chính đã hoàn thành. Ví dụ, khi VNPay xác nhận thanh toán thành công, Core Service không trực tiếp gửi email ngay trong request IPN. Thay vào đó, hệ thống cập nhật trạng thái đơn hàng, sau đó đẩy message vào RabbitMQ để worker xử lý cấp vé và gửi email. Cách thiết kế này giúp response trả về VNPay nhanh hơn, đồng thời giảm nguy cơ request bị timeout do các tác vụ tốn thời gian như tạo QR hoặc gửi email.

2.4. API Gateway và Service Discovery

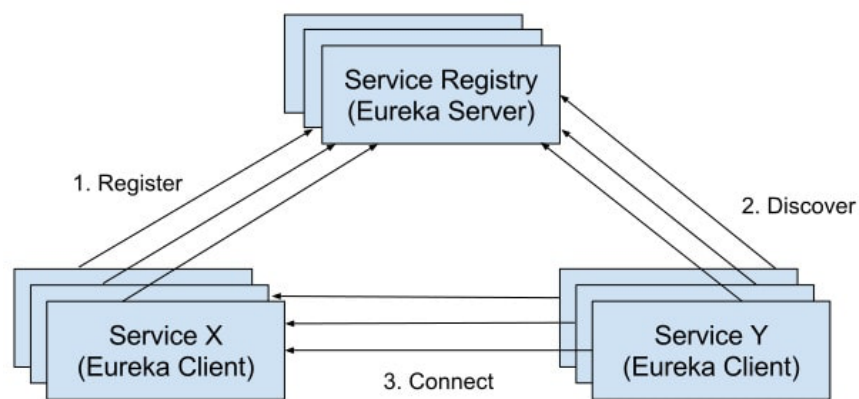
2.4.1. API Gateway



Hình 6. Giới thiệu API Gateway. Nguồn: blog.algomaster.io

API Gateway là điểm vào duy nhất của frontend khi gọi đến backend. Thay vì frontend phải biết địa chỉ của từng service, frontend chỉ cần gọi đến gateway. Gateway chịu trách nhiệm định tuyến request đến service phù hợp.

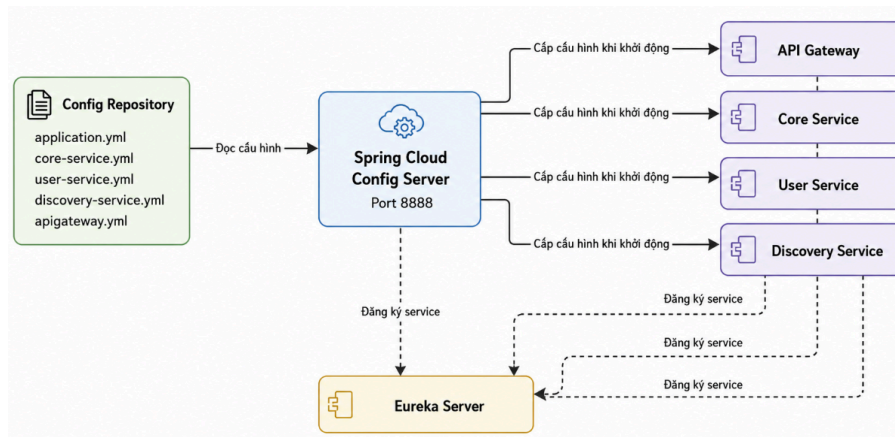
2.4.2. Eureka Service Discovery



Hình 7. Giới thiệu Eureka. Nguồn: dineshonjava.com

Trong kiến trúc microservice, địa chỉ của các service có thể thay đổi khi triển khai hoặc scale nhiều instance. Eureka Service Discovery giúp các service đăng ký thông tin của mình và tìm kiếm service khác thông qua tên logic thay vì địa chỉ cố định.

2.4.3. Spring Cloud Config Server

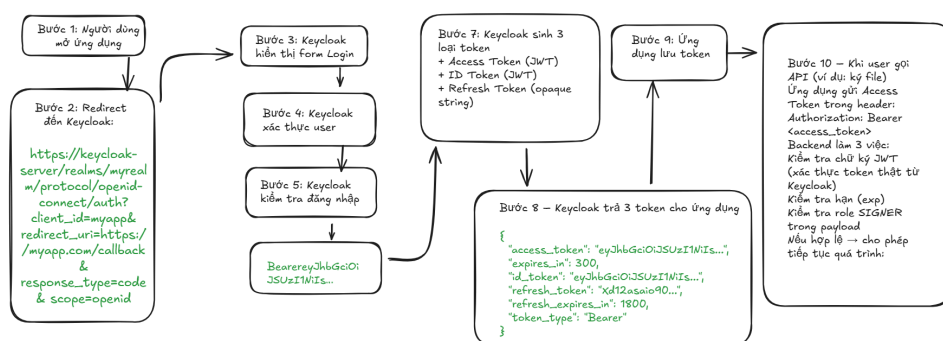


Hình 7. Spring Cloud Config Server

Spring Cloud Config Server được sử dụng để quản lý cấu hình tập trung cho các service. Thay vì mỗi service tự lưu toàn bộ cấu hình, các service có thể import cấu hình từ Config Server.

Cấu hình tập trung giúp dễ quản lý các thông tin như database URL, RabbitMQ, Redis, Keycloak, VNPay, Cloudinary và logging. Tuy nhiên, với môi trường production, các thông tin nhạy cảm cần được quản lý bằng biến môi trường hoặc secret manager để tránh lộ credential.

2.5. Xác thực và phân quyền bằng Keycloak, OAuth2 và JWT



Hình 8. Quy trình tích hợp Keycloak vào hệ thống ký số. Nguồn: viblo.asia

Hệ thống Flash Ticket sử dụng Keycloak để quản lý đăng nhập, xác thực và phân quyền người dùng. Keycloak đóng vai trò Identity Provider, cung cấp access token theo chuẩn OAuth2/OpenID Connect.

Sau khi người dùng đăng nhập thành công, frontend nhận JWT access token từ Keycloak. Khi gọi API, frontend gửi token này trong header:

Authorization: Bearer <access_token>

API Gateway và các backend service đều validate JWT. Việc kiểm tra ở cả Gateway và service tạo ra cơ chế bảo vệ nhiều lớp. Gateway chặn request không hợp lệ từ bên ngoài, còn service vẫn tự bảo vệ trong trường hợp bị gọi trực tiếp.

Keycloak cũng được mở rộng bằng custom RabbitMQ SPI. Khi có sự kiện đăng ký, cập nhật hoặc xóa user, Keycloak có thể gửi message qua RabbitMQ để User Service đồng bộ dữ liệu vào MongoDB. Ngoài ra, User Service còn có cơ chế self-healing từ JWT để tự tạo user nếu message đồng bộ bị trễ hoặc bị lỗi.

2.6. Cơ sở dữ liệu trong hệ Microservice

2.6.1. PostgreSQL cho dữ liệu nghiệp vụ



Hình 9. Giới thiệu PostgreSQL. Nguồn: viblo.asia

PostgreSQL được sử dụng làm cơ sở dữ liệu chính cho Core Service. Các dữ liệu nghiệp vụ như sự kiện, địa điểm, loại vé, đơn hàng, giao dịch, vé và khuyến mãi được lưu trong PostgreSQL. Trong database, dữ liệu được chia theo schema để phân tách miền chức năng:

Bảng 6. Các schema PostgreSQL chính trong hệ thống

Schema	Nhóm dữ liệu
event_schema	Sự kiện, địa điểm, danh mục, hình ảnh, loại vé
booking_schema	Đơn hàng, chi tiết đơn hàng, vé, reservation

payment_schema	Giao dịch thanh toán, hoàn tiền
promotion_schema	Mã khuyến mãi, lượt sử dụng
discovery_schema	Dữ liệu phục vụ chatbot và tìm kiếm

2.6.2. MongoDB cho dữ liệu người dùng



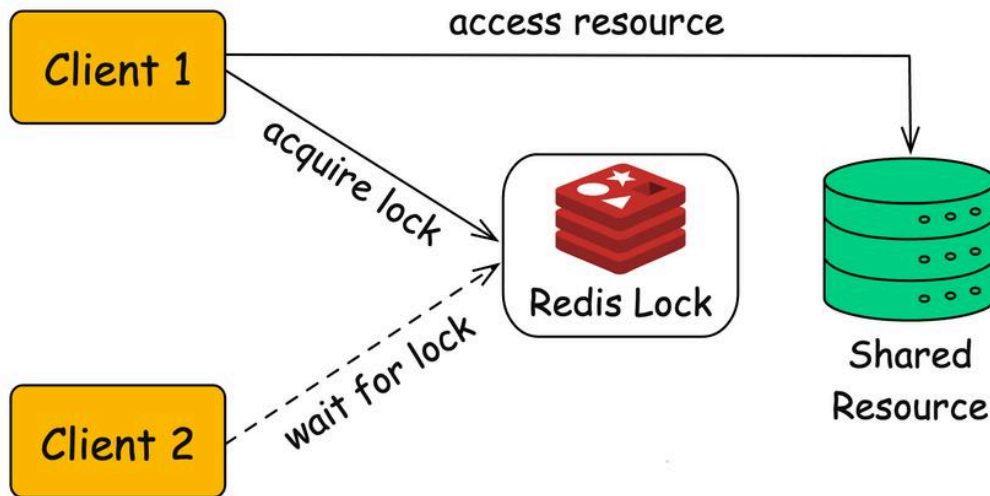
Hình 10. Giới thiệu MongoDB. Nguồn: viblo.aisa

User Service sử dụng MongoDB để lưu thông tin người dùng, hồ sơ organizer và quan hệ follow giữa người dùng với organizer. MongoDB phù hợp với dữ liệu có cấu trúc linh hoạt như profile, branding, contact, business info và preferences. Các collection chính gồm:

Bảng 7. Các collection MongoDB chính của User Service

Collection	Mục đích
users	Lưu thông tin user đồng bộ từ Keycloak
organizer_profiles	Lưu hồ sơ nhà tổ chức sự kiện
user_follows	Lưu quan hệ người dùng theo dõi organizer
user_activity_logs	Lưu log hoạt động người dùng

2.6.3. Redis cho distributed lock và chống xử lý trùng



Hình 11. Giới thiệu Redis cho distributed lock. Nguồn: viblo.asia

Redis được sử dụng trong hệ thống cho các bài toán cần tốc độ cao và tính nguyên tử, đặc biệt là khóa phân tán trong luồng đặt vé và thanh toán. Trong hệ thống Flash Ticket, Redis được dùng cho:

- Chống người dùng bấm đặt vé liên tục trong thời gian ngắn.
- Distributed lock theo loại vé để tránh oversell.
- Chống tạo nhiều link thanh toán cùng lúc.
- Idempotency lock khi xử lý IPN từ VNPay

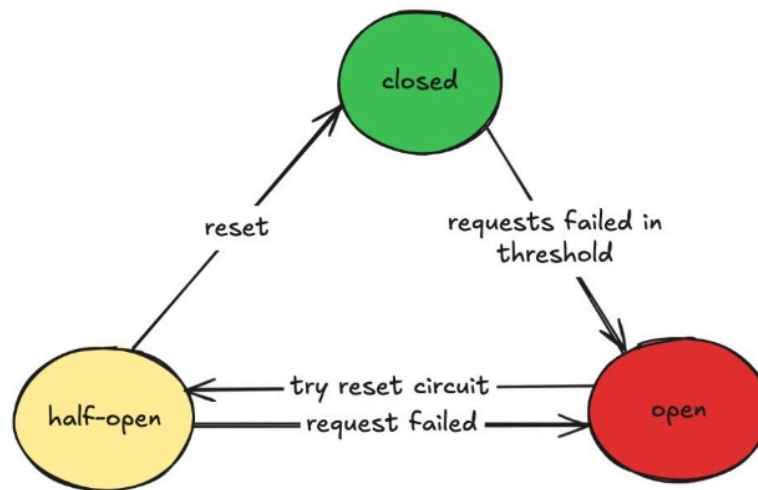
2.6.4. PGVector cho tìm kiếm ngữ nghĩa

PGVector là extension của PostgreSQL cho phép lưu trữ và tìm kiếm vector embedding. Trong Discovery Service, dữ liệu sự kiện được chuyển thành vector embedding và lưu vào bảng thuộc discovery_schema.

Khi người dùng hỏi chatbot hoặc tìm kiếm sự kiện bằng ngôn ngữ tự nhiên, hệ thống chuyển câu hỏi thành embedding rồi tìm các sự kiện có vector gần nhất. Cách tiếp cận này giúp hệ thống hỗ trợ tìm kiếm ngữ nghĩa tốt hơn so với tìm kiếm từ khóa truyền thống.

2.7. Một số kỹ thuật nâng cao trong hệ thống

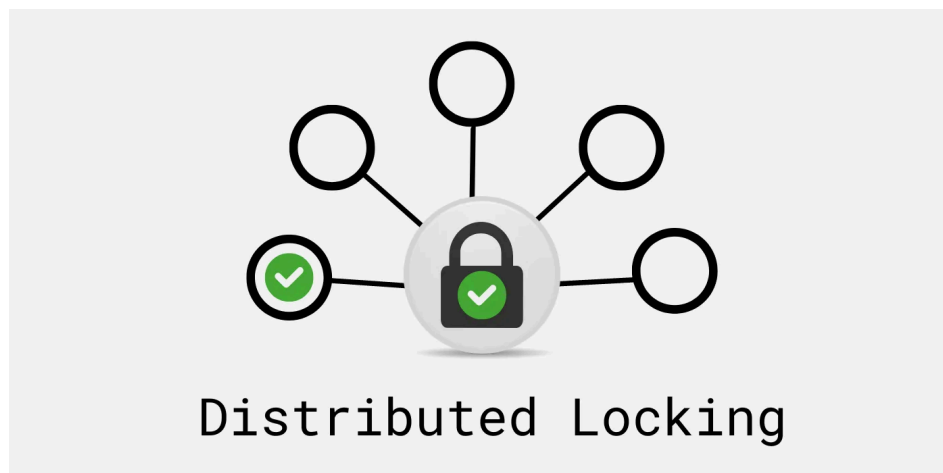
2.7.1. Circuit breaker và retry policy



Hình 12. Giới thiệu Circuit breaker và retry policy. Nguồn: dev.to

Circuit breaker là kỹ thuật giúp hệ thống tránh gọi liên tục vào một service đang lỗi. Khi số lần lỗi vượt ngưỡng, circuit breaker mở mạch và trả fallback nhanh thay vì tiếp tục gửi request đến service lỗi.

2.7.2. Distributed locking trong đặt vé



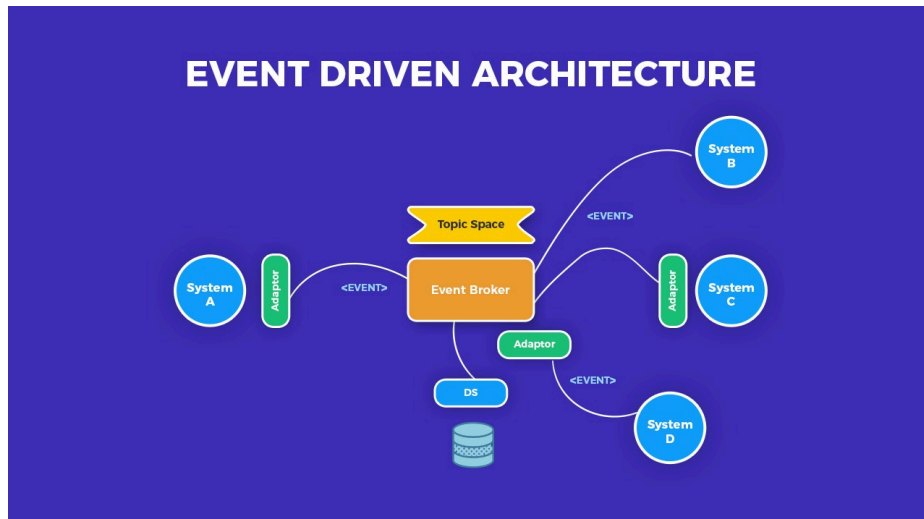
Hình 13. Giới thiệu Distributed locking. Nguồn: anhdh.net

Đặt vé là nghiệp vụ có rủi ro cao về cạnh tranh dữ liệu. Nếu nhiều người cùng mua một loại vé tại cùng thời điểm, hệ thống phải đảm bảo số lượng vé không bị bán vượt quá tồn kho. Hệ thống Flash Ticket sử dụng nhiều lớp bảo vệ:

Lớp bảo vệ	Vai trò
Redis spam lock	Chống một người gửi nhiều request đặt vé liên tục
Redisson distributed lock	Khóa theo ticket type hoặc từng ghế để giảm xung đột khi nhiều người đặt vé cùng lúc
Double-check stock	Kiểm tra lại tồn vé sau khi đã lấy lock
Atomic SQL update	Chỉ giảm tồn vé nếu số lượng còn đủ
Order expiration scheduler	Hoàn vé nếu đơn hàng hết hạn chưa thanh toán

Bảng 8. Các lớp bảo vệ trong luồng đặt vé

2.7.3. Event-driven workflow



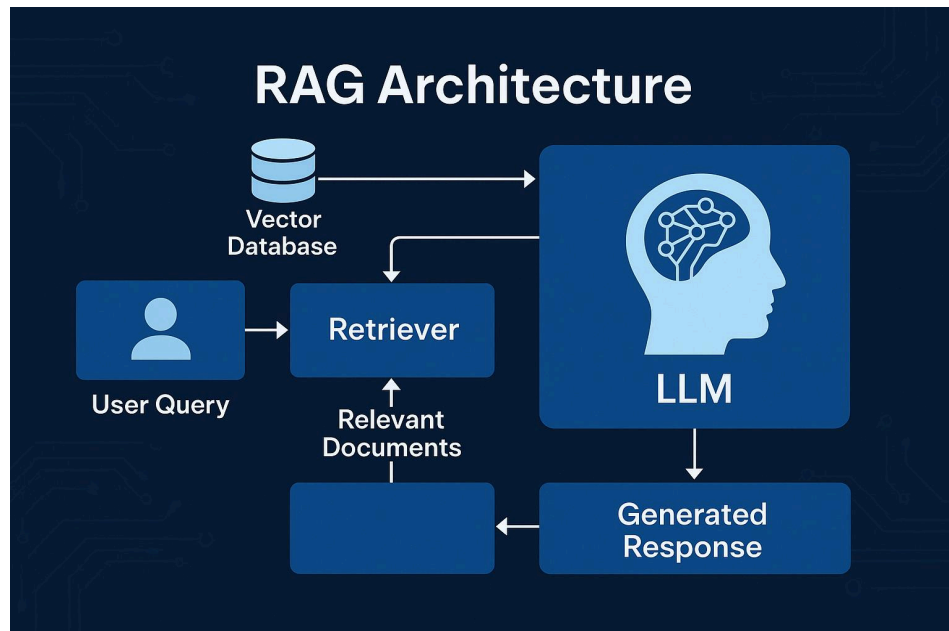
Hình 14. Giới thiệu Event-driven. Nguồn: linkedin.com

Event-driven workflow là cách thiết kế trong đó các service phát ra sự kiện khi có thay đổi nghiệp vụ quan trọng. Các service khác có thể lắng nghe sự kiện và xử lý phần việc của mình.

Trong hệ thống này, event-driven workflow được thể hiện ở luồng sau thanh toán và đồng bộ dữ liệu sự kiện. Khi VNPay xác nhận thanh toán thành công, Core Service cập nhật đơn hàng, phát hành vé theo cơ chế idempotent, sau đó phát PaymentSuccessEvent lên RabbitMQ sau khi đã thực hiện transaction commit. TicketMessageListener nhận event từ queue q.ticket.issue, lấy hoặc tạo vé theo cơ chế idempotent, thực hiện sinh và upload ảnh QR và phát tiếp TicketIssuedEvent.

EmailMessageListener nhận event từ queue q.email.send để gửi email xác nhận vé. Nếu gửi email lỗi, thanh toán không bị rollback và message có thể được đưa vào DLQ để xử lý sau.

2.7.4. RAG và chatbot hỗ trợ tìm kiếm sự kiện



Hình 15. Giới thiệu Retrieval-Augmented Generation. Nguồn: medium.com

RAG là viết tắt của Retrieval-Augmented Generation, là mô hình sinh câu trả lời có kết hợp truy xuất dữ liệu liên quan từ hệ thống. Discovery Service sử dụng LangChain4j, Gemini và PGVector để xây dựng chatbot hỗ trợ người dùng. Luồng xử lý cơ bản gồm:

- Bước 1. Người dùng gửi câu hỏi vào chatbot.
- Bước 2. Hệ thống phân loại câu hỏi bằng Adaptive RAG Router.
- Bước 3. Nếu cần tìm kiếm, hệ thống truy xuất sự kiện liên quan từ PGVector.
- Bước 4. LLM sử dụng dữ liệu truy xuất được để tạo câu trả lời.
- Bước 5. Nếu người dùng muốn đặt vé hoặc thanh toán, chatbot có thể gọi tool tương ứng để thực hiện qua Core Service.

2.8. Kết chương

Chương 2 đã trình bày các cơ sở lý thuyết và công nghệ chính được sử dụng trong hệ thống Flash Ticket. Nội dung bao gồm kiến trúc microservice, RESTful API, message queue, RabbitMQ, API Gateway, Eureka, Config Server, Keycloak, JWT, PostgreSQL, MongoDB, Redis, PGVector và các kỹ thuật nâng cao như circuit breaker, distributed locking, event-driven workflow và RAG chatbot.

Các nền tảng lý thuyết này là cơ sở để phân tích và thiết kế hệ thống ở chương tiếp theo. Trong Chương 3, báo cáo sẽ trình bày chi tiết kiến trúc tổng thể, thiết kế các microservice, thiết kế cơ sở dữ liệu, thiết kế API và các luồng xử lý chính của hệ thống Flash Ticket.

CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG FLASH TICKET

3.1. Phân tích yêu cầu hệ thống

3.1.1. Yêu cầu chức năng chính

Các yêu cầu chức năng chính được triển khai trong dự án gồm:

Bảng 9. Yêu cầu chức năng chính của hệ thống Flash Ticket

Nhóm chức năng	Nội dung
Quản lý sự kiện	Hệ thống cho phép hiển thị danh sách sự kiện, sự kiện nổi bật, chi tiết sự kiện, danh mục, địa điểm và sơ đồ ghế công khai.
Đặt vé	Người mua có thể chọn loại vé hoặc chọn ghế, tạo booking, xem đơn hàng, hủy đơn hàng chờ thanh toán và theo dõi trạng thái đơn.
Thanh toán	Hệ thống tạo URL thanh toán VNPay, xử lý IPN từ VNPay, cập nhật transaction, xác nhận order và phát hành vé sau thanh toán thành công.
Phát hành vé	Sau khi thanh toán thành công, hệ thống tạo vé, sinh QR data, upload QR image và gửi email xác nhận vé.
Quản lý organizer	Người dùng có thể gửi yêu cầu trở thành organizer; organizer quản lý hồ sơ, logo, banner, sự kiện, loại vé, layout và seat map.
Quản trị	Admin xem và duyệt hồ sơ đăng ký organizer.
Check-in	Organizer có giao diện và API check-in vé bằng QR.
Discovery và chatbot	Người dùng có thể tìm kiếm sự kiện bằng semantic search và sử dụng chatbot hỗ trợ tìm kiếm, đặt vé, thanh toán.

3.1.2. Yêu cầu kiến trúc Microservice

Hệ thống được phân tách thành nhiều service độc lập, mỗi service đảm nhận một miền chức năng riêng gồm:

Bảng 10. Phân tách trách nhiệm giữa các service

Thành phần	Vai trò trong hệ thống
API Gateway	Tiếp nhận request từ frontend, xác thực JWT, định tuyến đến các service, rate limit, retry và circuit breaker.
Core Service	Quản lý sự kiện, loại vé, seat map, booking, order, payment, ticket, QR, email và check-in.
User Service	Quản lý hồ sơ người dùng, organizer profile, follow organizer, đồng bộ user từ Keycloak.
Discovery Service	Xử lý semantic search, chatbot, RAG, embedding sự kiện và công cụ đặt vé/thanh toán qua chat.
Eureka Server	Cung cấp service discovery để các service đăng ký và được Gateway gọi qua logical service name.
Config Server	Quản lý cấu hình tập trung cho Gateway và các microservice.

3.1.3. Tác nhân sử dụng hệ thống

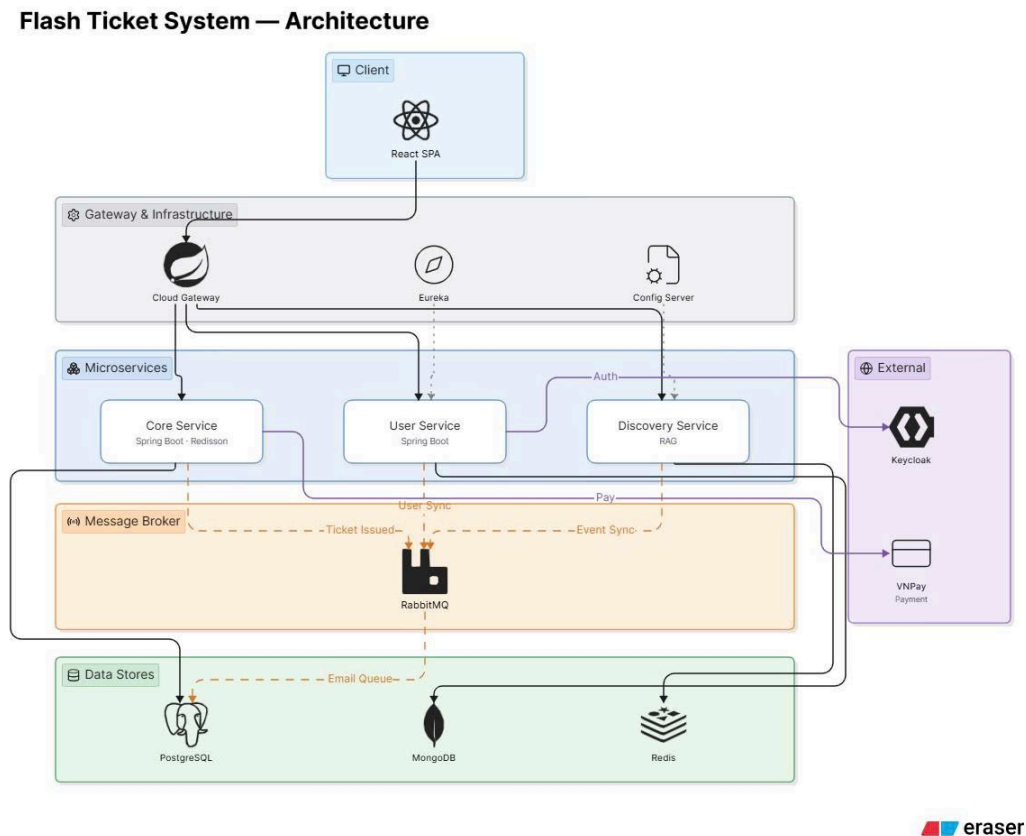
Các tác nhân chính của hệ thống được xác định theo chức năng gồm:

Bảng 11. Tác nhân sử dụng và tương tác với hệ thống Flash Ticket

Tác nhân	Chức năng sử dụng
Người mua vé	Xem sự kiện, tìm kiếm sự kiện, chọn vé/ghế, tạo booking, thanh toán, xem đơn hàng, xem vé QR, theo dõi organizer và sử dụng chatbot.
Nhà tổ chức	Gửi hồ sơ organizer, quản lý hồ sơ, tạo và cập nhật sự kiện, quản lý loại vé, thiết kế layout, publish seat map, upload media và check-in vé.
Quản trị viên	Xem danh sách hồ sơ organizer và duyệt hoặc từ chối hồ sơ đăng ký.

3.2. Thiết kế kiến trúc tổng thể hệ thống

3.2.1. Kiến trúc Microservice của Flash Ticket



Hình 16. Kiến trúc tổng thể hệ thống Flash Ticket

Hệ thống Flash Ticket được thiết kế theo mô hình microservice, trong đó hệ thống không gom toàn bộ chức năng vào một ứng dụng backend duy nhất mà tách thành nhiều service có trách nhiệm riêng.

Trong đó, kiến trúc backend được chia thành các service chính gồm API Gateway, Core Service, User Service, Discovery Service, Eureka Server và Config Server. Mỗi service được triển khai như một ứng dụng Spring Boot độc lập, có port chạy riêng và cấu hình riêng. Các service nghiệp vụ không được truy cập trực tiếp từ frontend mà được đặt sau API Gateway.

3.2.2. Phân tách trách nhiệm giữa các service

Việc phân tách trách nhiệm trong hệ thống Flash Ticket được thực hiện dựa trên miền nghiệp vụ. Core Service quản lý các nghiệp vụ cốt lõi của hệ thống đặt vé. User Service quản lý thông tin liên quan đến người dùng và nhà tổ chức. Discovery

Service xử lý phần tìm kiếm và chatbot. API Gateway không chứa nghiệp vụ đặt vé mà tập trung vào lớp truy cập, bảo mật và khả năng chịu lỗi.

Bảng 12. Phân tách trách nhiệm giữa các service

Thành phần	Ứng dụng
API Gateway	Định tuyến các nhóm API, xác thực OAuth2/JWT, ánh xạ role từ Keycloak, phân quyền theo endpoint, giới hạn request bằng Redis, cấu hình circuit breaker và fallback
Core Service	Quản lý sự kiện, danh mục, địa điểm, ticket type, layout, seat map, booking, order, payment, VNPay IPN, phát hành vé QR, gửi sự kiện RabbitMQ và check-in vé
User Service	Quản lý hồ sơ người dùng, hồ sơ organizer, API dành cho admin duyệt organizer, đồng bộ người dùng từ Keycloak thông qua RabbitMQ
Discovery Service	Nhận dữ liệu sự kiện từ RabbitMQ, tạo embedding, lưu dữ liệu vào PGVector, cung cấp API tìm kiếm và chatbot dùng LangChain4j
Eureka Server	Lưu danh sách instance đang chạy, giúp Gateway gọi service bằng logical service name
Config Server	Cung cấp cấu hình tập trung cho Gateway và các service như database, Redis, RabbitMQ, Eureka, Keycloak và Resilience4j

3.2.3. Giao tiếp đồng bộ bằng RESTful API

Giao tiếp đồng bộ được sử dụng cho các thao tác cần phản hồi ngay cho người dùng, chẳng hạn như xem danh sách sự kiện, xem chi tiết sự kiện, tạo booking, khởi tạo thanh toán, xem vé của tôi hoặc gửi câu hỏi cho chatbot. Trong các trường hợp này, frontend gửi request HTTP đến API Gateway và nhận response trực tiếp.

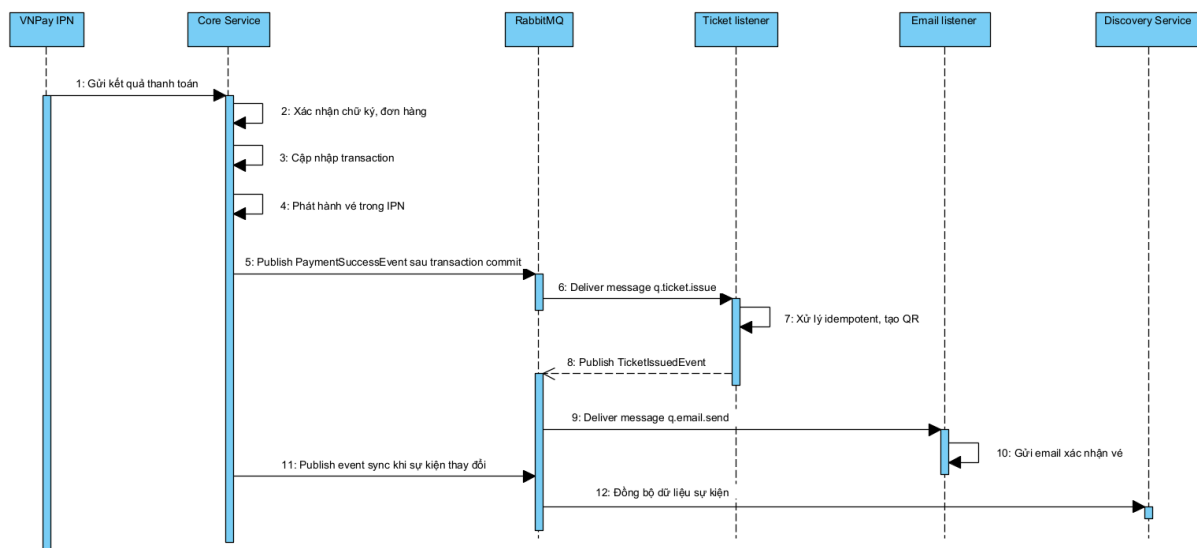
API Gateway định tuyến request theo nhóm đường dẫn. Các API liên quan đến sự kiện, booking, thanh toán và vé được chuyển đến Core Service. Các API liên quan đến người dùng và organizer được chuyển đến User Service. Các API tìm kiếm và chatbot được chuyển đến Discovery Service.

Bảng 13. Thiết kế định tuyến RESTful API qua API Gateway

Nhóm endpoint	Service xử lý	Mục đích
api/events	Core Service	Xem, quản lý và công bố sự kiện
api/bookings	Core Service	Tạo booking, giữ vé, xử lý đơn đặt vé
api/payments	Core Service	Khởi tạo thanh toán, nhận IPN từ VNPay, kiểm tra trạng thái thanh toán
api/orders	Core Service	Xem danh sách đơn hàng và chi tiết đơn hàng
api/tickets	Core Service	Xem vé, chi tiết vé, check-in vé
api/categories, api/venues	Core Service	Dữ liệu danh mục và địa điểm
api/layouts, api/seats, api/organizer	Core Service	Chức năng dành cho nhà tổ chức, gồm layout và seat map
api/users, api/organizers	User Service	Hồ sơ người dùng và hồ sơ organizer
api/admin/organizers	User Service	Quản trị viên duyệt hồ sơ organizer
api/discovery	Discovery Service	Tìm kiếm sự kiện
api/chat	Discovery Service	Chatbot hỗ trợ người dùng

3.2.4. Giao tiếp bất đồng bộ bằng RabbitMQ

RabbitMQ được sử dụng cho các nghiệp vụ không nên xử lý toàn bộ trong một request đồng bộ. Các luồng này thường có đặc điểm là cần tách bước xử lý, cần đảm bảo message không bị mất hoặc có thể xử lý lại khi lỗi. Trong hệ thống, RabbitMQ xuất hiện ở các luồng quan trọng như: phát hành vé sau thanh toán, gửi email xác nhận, đồng bộ dữ liệu sự kiện sang Discovery Service và đồng bộ người dùng từ Keycloak sang User Service.



Hình 17. Các luồng bất đồng bộ chính trong hệ thống Flash Ticket

Ở luồng thanh toán, sau khi VNPAY IPN được xác thực thành công, Core Service cập nhật trạng thái giao dịch và đơn hàng. Hệ thống hiện tại có cơ chế phát hành vé trong luồng xử lý IPN để đảm bảo đơn hàng đã thanh toán có vé tương ứng. Sau đó, sự kiện PaymentSuccessEvent vẫn được publish sau khi transaction commit để listener xử lý tiếp theo theo cơ chế idempotent. Listener phát hành vé kiểm tra vé đã tồn tại hay chưa, tạo QR cho vé và phát tiếp TicketIssuedEvent để email listener gửi email xác nhận.

3.3. Thiết kế các Microservice trong hệ thống

3.3.1. API Gateway

API Gateway là điểm truy cập đầu tiên của toàn bộ backend. Frontend không gọi trực tiếp Core Service, User Service hoặc Discovery Service mà gửi request đến Gateway. Sau đó, Gateway dựa vào đường dẫn API để chuyển request đến service phù hợp thông qua tên service đã đăng ký trên Eureka. Trong hệ thống, API Gateway đảm nhận các nhiệm vụ chính sau:

Bảng 14. Các nhóm route chính của API Gateway

Nhóm nhiệm vụ	Mô tả
Định tuyến request	Chuyển request từ frontend đến Core Service, User Service hoặc Discovery Service
Xác thực	Cấu hình OAuth2 Resource Server để kiểm tra JWT do Keycloak phát hành
Phân quyền	Kiểm tra quyền truy cập theo role như BUYER, ORGANIZER và ADMIN
Rate limiting	Sử dụng RedisRateLimiter để giới hạn tần suất request
Chịu lỗi	Cấu hình circuit breaker cho Core Service, User Service và Discovery Service
Retry	Retry cho một số request GET đến Core Service
Fallback	Trả response fallback khi service phía sau không phản hồi hoặc gặp lỗi

API Gateway không xử lý nghiệp vụ đặt vé hay thanh toán, mà đóng vai trò lớp bảo vệ và điều phối request. Nhờ đó, các service phía sau có thể tập trung vào nghiệp vụ riêng, còn các vấn đề chung như xác thực, phân quyền, rate limit và fallback được gom về một điểm thống nhất.

3.3.2. Core Service

Core Service là service trung tâm của hệ thống vì chứa hầu hết nghiệp vụ liên quan trực tiếp đến sự kiện và đặt vé. Service này sử dụng PostgreSQL để lưu dữ liệu nghiệp vụ như sự kiện, địa điểm, danh mục, loại vé, layout, seat map, đơn hàng, giao dịch thanh toán và vé đã phát hành. Các nhóm chức năng chính của Core Service gồm:

Bảng 15. Các nhóm chức năng chính của Core Service

Nhóm chức năng	Nội dung xử lý
Quản lý sự kiện	Tạo, cập nhật, công bố, hủy công bố và hiển thị sự kiện
Quản lý ticket type	Thiết lập loại vé, giá vé, số lượng, thời gian mở bán và giới hạn mua
Layout và seat map	Quản lý sơ đồ chỗ ngồi, khu vực, ghế và trạng thái ghế
Booking	Tạo đơn đặt vé, kiểm tra tồn vé, giữ vé hoặc giữ ghế
Thanh toán	Khởi tạo thanh toán VNPay, xử lý VNPay IPN và cập nhật giao dịch
Phát hành vé	Tạo vé điện tử, sinh dữ liệu QR và cập nhật trạng thái vé
Check-in	Kiểm tra vé tại sự kiện
Đồng bộ sự kiện	Gửi sự kiện thay đổi sang RabbitMQ để Discovery Service cập nhật dữ liệu tìm kiếm

Core Service có nhiều cơ chế bảo vệ cho nghiệp vụ đặt vé. Khi người dùng tạo booking, hệ thống kiểm tra trạng thái sự kiện, thời gian mở bán, giới hạn số lượng mua và tồn kho vé. Với loại vé theo ghế, hệ thống kiểm tra danh sách ghế được chọn. Với loại vé theo số lượng, hệ thống giảm tồn vé bằng cập nhật có điều kiện để tránh bán vượt quá số lượng còn lại.

Redis và Redisson được sử dụng trong Core Service để giảm rủi ro xử lý trùng. Hệ thống có spam lock khi tạo booking, spam lock khi khởi tạo thanh toán, idempotency lock khi nhận VNPay IPN, distributed lock theo ticket type và seat, đồng thời có cache trạng thái ghế phục vụ seat map.

3.3.3. User Service

User Service phụ trách dữ liệu người dùng và nhà tổ chức sự kiện. Service này tách khỏi Core Service để phân quản lý định danh, hồ sơ và organizer không bị trộn với nghiệp vụ đặt vé. Trong hệ thống này, User Service sử dụng MongoDB để lưu dữ liệu người dùng và organizer. Ngoài các API dành cho người dùng cuối, service này còn có API dành cho quản trị viên để duyệt hồ sơ organizer.

Bảng 16. Các nhóm chức năng chính của User Service

Nhóm chức năng	Nội dung xử lý
Hồ sơ người dùng	Lưu và cập nhật thông tin người dùng trong hệ thống
Hồ sơ organizer	Lưu thông tin đăng ký trở thành nhà tổ chức sự kiện
Duyệt organizer	Cho phép quản trị viên xem, duyệt hoặc từ chối hồ sơ organizer
Đồng bộ Keycloak	Nhận sự kiện người dùng từ RabbitMQ để đồng bộ dữ liệu với Keycloak
Tự phục hồi dữ liệu JWT	Bổ sung hoặc cập nhật dữ liệu người dùng khi phát hiện JWT hợp lệ nhưng dữ liệu nội bộ chưa đầy đủ

3.3.4. Discovery Service

Discovery Service là service phục vụ tìm kiếm sự kiện và chatbot. Service này được tách riêng vì các chức năng tìm kiếm ngữ nghĩa, embedding và RAG có đặc điểm xử lý khác với nghiệp vụ đặt vé.

Discovery Service nhận dữ liệu sự kiện từ Core Service thông qua RabbitMQ. Khi sự kiện được tạo, cập nhật hoặc xóa, Core Service phát message đồng bộ sang queue q.discovery.event.sync. Discovery Service nhận message này, sau đó cập nhật dữ liệu tìm kiếm và embedding tương ứng.

Bảng 17. Các nhóm chức năng chính của Discovery Service

Nhóm chức năng	Nội dung xử lý
Tìm kiếm sự kiện	Cung cấp API tìm kiếm sự kiện theo truy vấn của người dùng
Đồng bộ dữ liệu	Nhận message từ RabbitMQ để cập nhật dữ liệu sự kiện
Embedding	Tạo vector embedding cho dữ liệu sự kiện
PGVector	Lưu embedding trong PostgreSQL có hỗ trợ PGVector
RAG	Truy xuất dữ liệu liên quan trước khi tạo câu trả lời

Chatbot	Hỗ trợ người dùng tìm kiếm sự kiện và thao tác qua công cụ tích hợp
---------	---

Discovery Service sử dụng LangChain4j, Gemini và PGVector. Thành phần Adaptive RAG Router phân loại truy vấn để chọn cách xử lý phù hợp, ví dụ trả lời trực tiếp, tìm kiếm đơn giản, truy vấn nhiều bước hoặc xử lý lại truy vấn chưa rõ ràng. Chatbot có thể sử dụng ngữ cảnh người dùng và JWT để gọi các công cụ phù hợp khi người dùng cần thao tác liên quan đến sự kiện, đặt vé hoặc thanh toán.

3.3.5. Eureka Server và Config Server

Eureka Server và Config Server là hai service hạ tầng quan trọng trong kiến trúc của hệ thống Flash Ticket. Hai service này không xử lý nghiệp vụ đặt vé, nhưng giúp các microservice hoạt động ổn định hơn trong môi trường phân tán.

Eureka Server đóng vai trò service discovery. Khi Core Service, User Service, Discovery Service và API Gateway khởi động, các service đăng ký thông tin instance của mình lên Eureka. API Gateway sau đó có thể định tuyến request đến các service bằng logical service name.

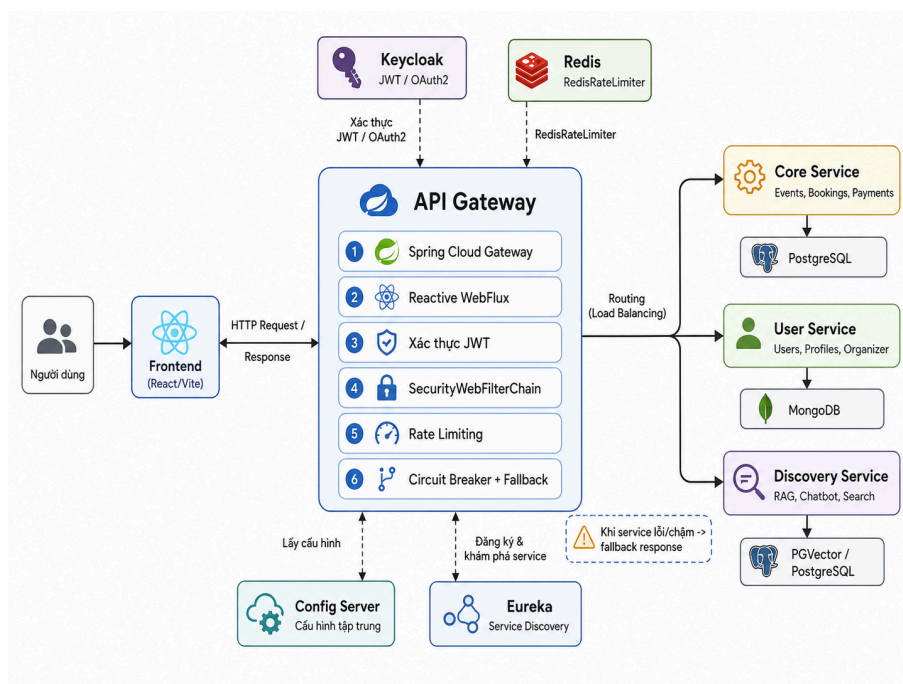
Config Server đóng vai trò quản lý cấu hình tập trung. Các service lấy cấu hình từ Config Server khi khởi động. Cấu hình có thể bao gồm thông tin kết nối database, Redis, RabbitMQ, Keycloak, Eureka và Resilience4j.

Bảng 18. Vai trò của Eureka Server và Config Server

Thành phần	Vai trò	Ý nghĩa trong hệ thống
Eureka Server	Service registry	Giúp các service đăng ký và phát hiện nhau
API Gateway kết hợp Eureka	Load-balanced routing	Cho phép Gateway gọi service theo tên thay vì địa chỉ cố định
Config Server	Centralized configuration	Tập trung cấu hình của nhiều service tại một nơi
Config files theo service	Cấu hình riêng từng service	Giúp mỗi service có cấu hình độc lập nhưng vẫn được quản lý thống nhất

Nhờ Eureka Server, hệ thống có thể mở rộng số lượng instance của một service mà không cần thay đổi cách frontend gọi API. Và nhờ Config Server, cấu hình của các service được quản lý tập trung, giảm việc lặp cấu hình trong từng module và thuận lợi hơn khi triển khai nhiều môi trường

3.4. Thiết kế API Gateway



Hình 18. API Gateway trong hệ thống Flash Ticket

API Gateway là lớp trung gian giữa frontend và các microservice phía sau. Trong hệ thống Flash Ticket, frontend không gọi trực tiếp Core Service, User Service hoặc Discovery Service mà gửi toàn bộ request đến API Gateway. Gateway sau đó thực hiện kiểm tra bảo mật, áp dụng một số chính sách kỹ thuật và chuyển tiếp request đến service phù hợp.

API Gateway được xây dựng bằng Spring Cloud Gateway, chạy theo mô hình reactive WebFlux. Vì vậy phần bảo mật tại Gateway cũng sử dụng SecurityWebFilterChain thay vì cơ chế servlet truyền thống. Gateway vừa đóng vai trò định tuyến, vừa là lớp kiểm soát truy cập đầu tiên trước khi request đi vào các service nghiệp vụ.

3.4.1. Vai trò của API Gateway

Trong kiến trúc Flash Ticket, API Gateway có vai trò tập trung các xử lý chung trước khi request được đưa vào service phía sau. Các xử lý này không thuộc riêng nghiệp vụ đặt vé, nhưng cần được áp dụng thống nhất cho nhiều nhóm API.

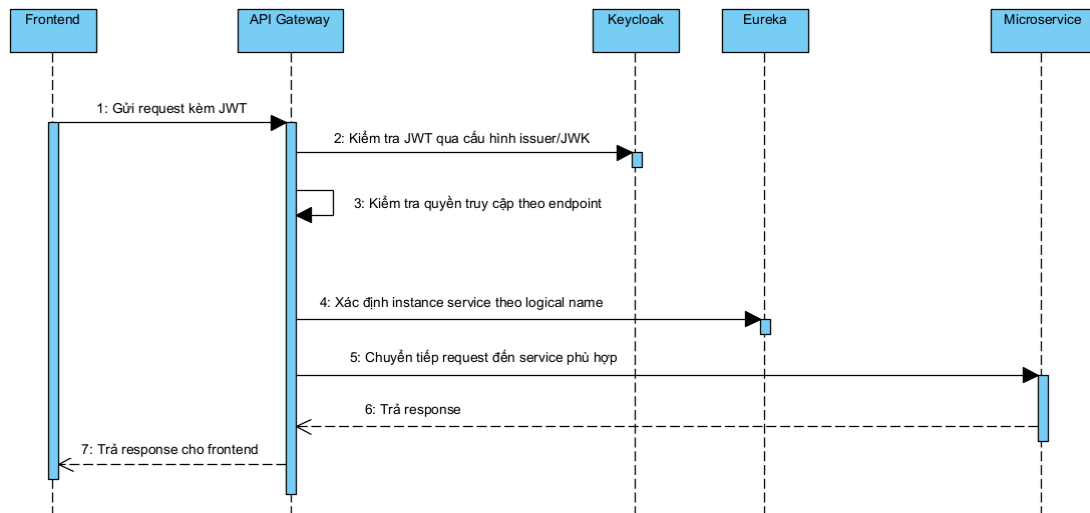
Bảng 19. Vai trò của API Gateway trong hệ thống Flash Ticket

Vai trò	Mô tả triển khai
Điểm vào tập trung	Frontend gửi request đến API Gateway thay vì gọi trực tiếp từng microservice
Định tuyến request	Gateway định tuyến request đến Core Service, User Service hoặc Discovery Service dựa trên path
Xác thực JWT	Gateway được cấu hình như OAuth2 Resource Server để kiểm tra access token từ Keycloak
Phân quyền	Gateway đọc role từ realm_access.roles trong JWT và chuyển thành ROLE_BUYER, ROLE_ORGANIZER, ROLE_ADMIN
Rate limiting	Gateway dùng RedisRateLimiter để giới hạn request theo host/IP người gửi
Circuit breaker	Gateway cấu hình circuit breaker cho Core Service, User Service và Discovery Service
Retry	Gateway retry tối đa 3 lần cho request GET đến Core Service
Fallback	Gateway trả response dự phòng khi service phía sau chậm hoặc không phản hồi
CORS	Gateway cho phép frontend chạy ở localhost:5173 và localhost:3000 gọi API

Với thiết kế này, API Gateway giúp giảm lặp logic bảo mật ở frontend và tránh để frontend phụ thuộc trực tiếp vào địa chỉ triển khai của từng service. Khi cần thay đổi địa chỉ service, tăng số lượng instance hoặc thêm cơ chế chịu lỗi, hệ thống có thể xử lý tại Gateway mà không cần thay đổi cách frontend gọi API.

3.4.2. Định tuyến request đến các microservice

API Gateway định tuyến request dựa trên nhóm đường dẫn API. Trong hệ thống, các route được khai báo trong cấu hình RouteLocator. Mỗi route gồm ba phần chính: điều kiện path, các filter áp dụng cho route và URI của service đích.



Hình 19. Luồng xử lý request qua API Gateway

3.4.3. Load-balanced route thông qua Eureka

Trong các route đến Core Service, User Service và Discovery Service, Gateway không dùng URL cố định dạng `http://localhost:8081`. Thay vào đó, hệ thống sử dụng URI dạng `lb://SERVICE-NAME`

Cách viết này thể hiện việc Gateway gọi service thông qua cơ chế load-balanced route của Spring Cloud. Gateway sẽ dựa vào danh sách instance lấy từ Eureka để xác định service đích đang hoạt động. Nhờ đó, Gateway không cần biết trước địa chỉ cụ thể của từng instance.

Khi các service khởi động, chúng đăng ký thông tin instance lên Eureka Server. API Gateway cũng được cấu hình là Eureka Client, có `register-with-eureka: true` và `fetch-registry: true`. Điều này cho phép Gateway vừa đăng ký chính nó vào Eureka, vừa lấy danh sách các service khác để định tuyến request.

Thiết kế này có ý nghĩa quan trọng trong hệ thống microservice. Nếu một service được triển khai nhiều instance, Gateway có thể phân phối request đến các

instance đang được Eureka quản lý. Nếu địa chỉ triển khai của service thay đổi, chỉ cần service đăng ký lại với Eureka, Gateway vẫn có thể gọi service bằng logical service name.

3.4.4. Xác thực và phân quyền tại Gateway

API Gateway là lớp kiểm tra bảo mật đầu tiên của hệ thống. Hệ thống cấu hình Gateway như một OAuth2 Resource Server, sử dụng JWT do Keycloak phát hành. Khi request gửi đến Gateway có header **Authorization: Bearer <token>**, Gateway kiểm tra token dựa trên issuer-uri và jwk-set-uri được cấu hình trong Config Server.

Sau khi JWT hợp lệ, Gateway trích xuất role từ claim realm_access.roles. Hệ thống không sử dụng cách đọc authority mặc định của Spring Security, mà có converter riêng để lấy các role nghiệp vụ BUYER, ORGANIZER và ADMIN, sau đó chuyển thành authority tương ứng là ROLE_BUYER, ROLE_ORGANIZER và ROLE_ADMIN.

3.4.5. Rate limiting bằng Redis

API Gateway sử dụng RedisRateLimiter để giới hạn tần suất request. Trong đó, RedisRateLimiter được cấu hình với các tham số 10, 20, 1. Điều này tương ứng với tốc độ nạp token, sức chứa tối đa của bucket và số token tiêu thụ cho mỗi request.

Bảng 20. Cấu hình RedisRateLimiter tại Gateway

Tham số	Giá trị	Ý nghĩa
Replenish rate	10	Số token được nạp lại mỗi giây
Burst capacity	20	Số token tối đa có thể tích lũy
Requested tokens	1	Số token tiêu thụ cho mỗi request

Gateway sử dụng KeyResolver dựa trên host name của địa chỉ gửi request. Theo đó, giới hạn request được áp dụng theo từng nguồn gửi request thay vì áp dụng chung cho toàn hệ thống.

Trong cấu hình hiện tại, filter requestRateLimiter đang được gắn trên route core-service. Điều này có nghĩa là các request đi vào nhóm API của Core Service như

sự kiện, booking, payment, order, ticket, layout và seat map sẽ chịu giới hạn tần suất tại Gateway. Đây là nhóm route quan trọng vì chứa nhiều nghiệp vụ có rủi ro bị spam, đặc biệt là booking và payment.

3.4.6. Fallback khi service không phản hồi

Để tránh việc frontend phải chờ quá lâu hoặc nhận lỗi không rõ ràng khi service phía sau gặp sự cố, API Gateway cấu hình circuit breaker và fallback cho từng nhóm service chính. Khi Core Service, User Service hoặc Discovery Service chậm, lỗi hoặc không phản hồi trong điều kiện circuit breaker, Gateway sẽ forward request đến endpoint fallback tương ứng.

Bảng 21. Fallback được cấu hình tại API Gateway

Service	Circuit breaker name	Fallback URI	Response khi lỗi
Core Service	coreServiceBreaker	forward:/fallback/core	503 Service Unavailable kèm thông báo Core Service chậm hoặc không khả dụng
User Service	userServiceBreaker	forward:/fallback/user	503 Service Unavailable kèm thông báo User Service chậm hoặc không khả dụng
Discovery Service	discoveryServiceBreaker	forward:/fallback/discovery	503 Service Unavailable kèm thông báo Discovery Service chậm hoặc không khả dụng

Fallback không thay thế cho việc xử lý lỗi nghiệp vụ trong từng service. Vai trò của fallback tại Gateway là trả về phản hồi rõ ràng khi service phía sau không sẵn sàng, từ đó giúp frontend có thể hiển thị thông báo phù hợp và tránh để request treo

quá lâu. Trong Flash Ticket, cơ chế này đặc biệt cần thiết vì hệ thống có các luồng nhạy cảm như đặt vé, thanh toán và chatbot, nơi người dùng cần nhận được phản hồi rõ ràng khi service tạm thời không hoạt động

3.5. Thiết kế khả năng chịu lỗi với Resilience4j

Trong hệ thống, khả năng chịu lỗi được triển khai tại API Gateway bằng Resilience4j thông qua Spring Cloud Gateway. Cơ chế này giúp Gateway hạn chế việc tiếp tục gọi đến một service đang lỗi hoặc phản hồi chậm, đồng thời trả về phản hồi fallback rõ ràng cho frontend.

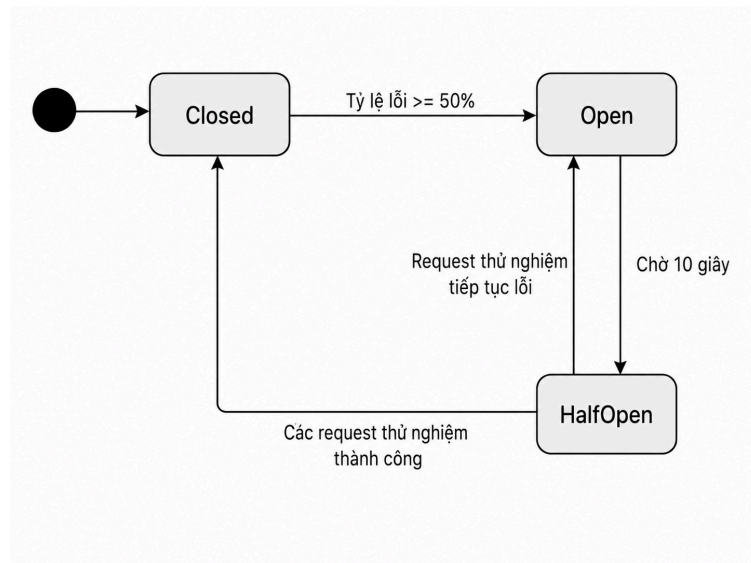
3.5.1. Circuit breaker trong API Gateway

Circuit breaker là cơ chế theo dõi kết quả các request gần nhất đến một service. Nếu tỷ lệ lỗi vượt ngưỡng cấu hình, circuit breaker chuyển sang trạng thái mở và Gateway không tiếp tục gửi request đến service lỗi trong một khoảng thời gian nhất định. Thay vào đó, request được chuyển đến fallback endpoint. Trong API Gateway của Flash Ticket, circuit breaker được gắn vào từng route chính:

Bảng 22. Circuit breaker được cấu hình tại API Gateway

Route	Service đích	Circuit breaker
core-service	lb://CORE-SERVICE	coreServiceBreaker
user-service	lb://USER-SERVICE	userServiceBreaker
discovery-service	lb://DISCOVERY-SERVICE	discoveryServiceBreaker

Cơ chế circuit breaker trong Gateway giúp cô lập lỗi theo từng service. Nếu Discovery Service bị chậm do xử lý chatbot, lỗi này không làm ảnh hưởng trực tiếp đến route của Core Service hoặc User Service. Tương tự, nếu User Service không phản hồi, các API đặt vé ở Core Service vẫn có thể tiếp tục hoạt động nếu Core Service vẫn ổn định



Hình 20. Trạng thái circuit breaker trong API Gateway

3.5.2. Circuit breaker cho Core Service, User Service và Discovery Service

Ba circuit breaker có cấu hình chung về ngưỡng lỗi, kích thước cửa sổ theo dõi và trạng thái half-open. Các cấu hình này được đặt trong file cấu hình của Gateway trên Config Server.

Bảng 23. Cấu hình circuit breaker trong Gateway

Thuộc tính	Giá trị	Ý nghĩa
slidingWindowSize	10	Theo dõi 10 lần gọi gần nhất
minimumNumberOfCalls	5	Cần ít nhất 5 lần gọi trước khi tính tỷ lệ lỗi
failureRateThreshold	50	Nếu tỷ lệ lỗi từ 50% trở lên thì mở circuit
waitDurationInOpenState	10s	Circuit ở trạng thái open trong 10 giây trước khi thử lại
permittedNumberOfCallsInHalfOpenState	3	Cho phép 3 request thử nghiệm ở trạng thái half-open
automaticTransitionFromOpenToHalfOpenEnabled	true	Tự chuyển từ open sang half-open sau thời gian chờ

slidingWindowType	count_based	Tính cửa sổ theo số lượng request
registerHealthIndicator	true	Đưa trạng thái circuit breaker vào actuator health

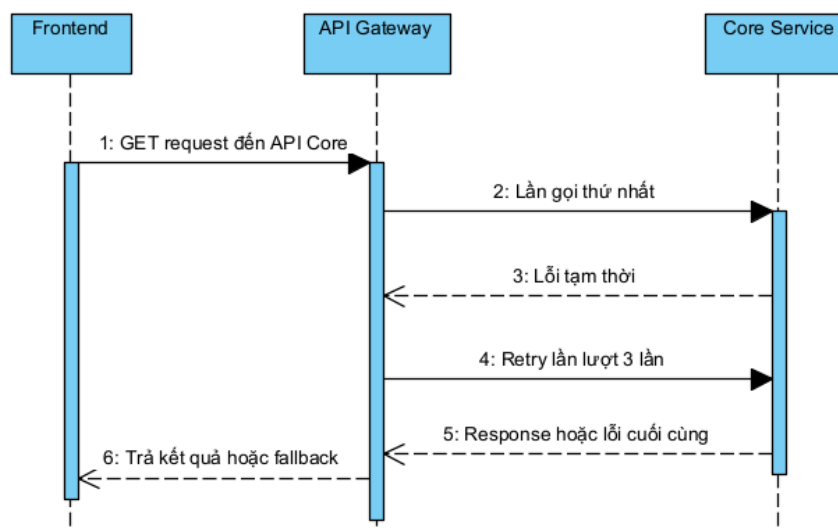
Với cấu hình này, Gateway không ngắt mạch ngay khi có một lỗi đơn lẻ. Hệ thống chỉ đánh giá circuit breaker khi đã có tối thiểu 5 lần gọi. Sau đó, kết quả được tính trên cửa sổ 10 lần gọi gần nhất. Nếu tỷ lệ lỗi đạt từ 50% trở lên, circuit breaker mở và request tiếp theo sẽ đi vào fallback.

Đối với Core Service, circuit breaker bảo vệ các API quan trọng như sự kiện, booking, payment, order và ticket. Đối với User Service, cơ chế này bảo vệ các API hồ sơ người dùng, organizer và admin organizer. Đối với Discovery Service, circuit breaker giúp tránh để frontend chờ quá lâu khi chức năng tìm kiếm hoặc chatbot gặp lỗi.

3.5.3. Retry policy cho request GET

Ngoài circuit breaker, Gateway còn cấu hình retry cho route core-service. Cơ chế retry chỉ áp dụng cho request GET và số lần retry là 3. Việc chỉ retry request GET là phù hợp với tính chất idempotent của phương thức này. Các request GET thường dùng để đọc dữ liệu như danh sách sự kiện, chi tiết sự kiện, danh mục, địa điểm hoặc thông tin vé. Nếu service gặp lỗi tạm thời, Gateway có thể thử lại mà ít rủi ro tạo ra tác dụng phụ.

Ngược lại, hệ thống không cấu hình retry cho các request ghi dữ liệu như POST, PUT, PATCH hoặc DELETE. Đây là lựa chọn quan trọng trong hệ thống đặt vé, vì retry tự động một request tạo booking hoặc thanh toán có thể làm tăng nguy cơ xử lý trùng nếu không được kiểm soát chặt chẽ ở tầng nghiệp vụ



Hình 21. Retry policy cho request GET đến Core Service

3.5.4. Time limiter cho các service

Time limiter được dùng để giới hạn thời gian Gateway chờ phản hồi từ service phía sau. Nếu service xử lý quá lâu vượt quá thời gian cấu hình, request có thể được xem là thất bại và đi vào cơ chế fallback.

Bảng 24. Cấu hình time limiter cho các service

Circuit breaker	Timeout	Lý do thiết kế
coreServiceBreaker	10s	Core Service xử lý nghiệp vụ chính như sự kiện, booking, payment và ticket
userServiceBreaker	10s	User Service xử lý hồ sơ người dùng và organizer
discoveryServiceBreaker	60s	Discovery Service có chatbot, quá trình trả lời có thể lâu hơn API thông thường

Discovery Service được cấu hình timeout dài hơn Core Service và User Service vì chatbot có thể cần thời gian để xử lý truy vấn, truy xuất dữ liệu liên quan và gọi mô hình ngôn ngữ. Tuy nhiên, việc đặt timeout vẫn cần thiết để tránh request bị treo vô thời hạn.

3.5.5. Ý nghĩa của cơ chế chịu lỗi trong hệ thống đặt vé

Đối với hệ thống đặt vé sự kiện, khả năng chịu lỗi có ý nghĩa quan trọng vì các thao tác của người dùng thường diễn ra trong thời gian ngắn và có tính cạnh tranh cao. Nếu một service bị chậm hoặc lỗi, hệ thống cần phản hồi rõ ràng thay vì để request treo hoặc tiếp tục dồn tải vào service đang gặp sự cố. Trong hệ thống Resilience4j tại API Gateway mang lại các lợi ích chính sau:

Bảng 25. Ý nghĩa của Resilience4j trong Flash Ticket

Cơ chế	Ý nghĩa đối với hệ thống
Circuit breaker	Tránh tiếp tục gọi liên tục vào service đang lỗi, giảm tải cho service phía sau
Retry cho GET	Tăng khả năng thành công cho các request đọc dữ liệu khi lỗi chỉ xảy ra tạm thời
Time limiter	Giới hạn thời gian chờ, tránh request treo quá lâu
Fallback	Trả response rõ ràng khi service không phản hồi
Cấu hình riêng theo service	Cho phép điều chỉnh timeout và circuit breaker theo đặc điểm từng service

3.6. Thiết kế Service Discovery bằng Eureka

Trong kiến trúc microservice, các service có thể được triển khai độc lập và có địa chỉ thay đổi theo môi trường chạy. Nếu API Gateway phải gọi trực tiếp từng service bằng địa chỉ cố định, hệ thống sẽ khó mở rộng và khó thay đổi khi service được triển khai nhiều instance. Vì vậy, Flash Ticket sử dụng Eureka Server để thực hiện service discovery.

Service discovery giúp các service đăng ký thông tin hoạt động của mình vào một registry chung. API Gateway sau đó không cần biết địa chỉ cụ thể của từng service, mà có thể gọi service thông qua logical service name như CORE-SERVICE, USER-SERVICE hoặc DISCOVERY-SERVICE.

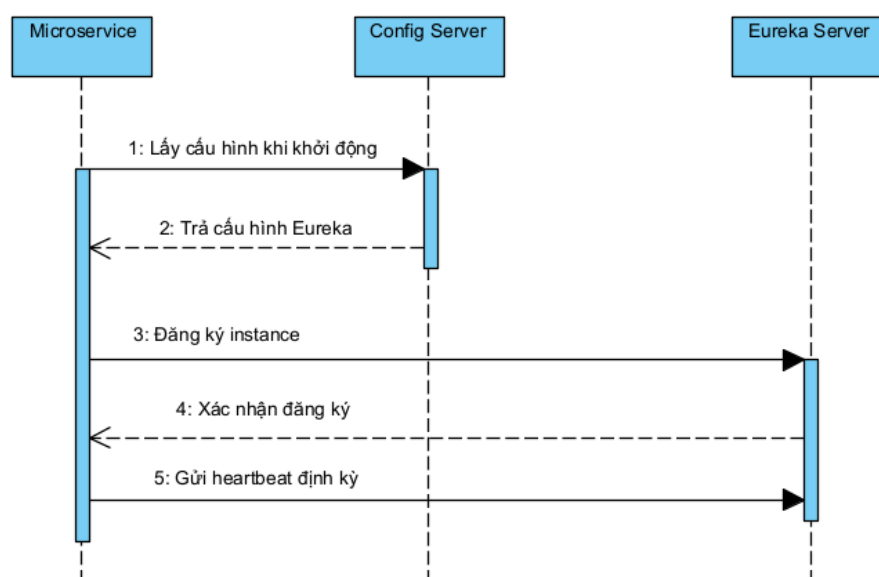
3.6.1. Vai trò của Eureka Server

Eureka Server trong Flash Ticket đóng vai trò service registry. Đây là nơi lưu danh sách các service đang chạy, bao gồm tên service, địa chỉ, port và trạng thái hoạt động. Các service như API Gateway, Core Service, User Service và Discovery Service đều được cấu hình là Eureka Client.

Bảng 26. Vai trò của Eureka Server trong hệ thống

Vai trò	Mô tả
Service registry	Lưu danh sách các service đã đăng ký
Theo dõi trạng thái service	Nhận heartbeat để biết service còn hoạt động hay không
Hỗ trợ định tuyến qua Gateway	Cho phép Gateway gọi service bằng logical service name
Giảm phụ thuộc địa chỉ tĩnh	Gateway không cần cấu hình cứng địa chỉ IP hoặc port của từng service
Hỗ trợ mở rộng service	Khi có nhiều instance, registry có thể cung cấp danh sách instance cho Gateway

3.6.2. Cơ chế service đăng ký vào Eureka



Hình 22. Cơ chế service đăng ký vào Eureka

Khi một service khởi động, service đó đọc cấu hình Eureka từ Config Server. Sau đó, service gửi thông tin đăng ký đến Eureka Server. Thông tin đăng ký bao gồm tên ứng dụng, địa chỉ instance và port đang chạy.

Các service ứng dụng như Core Service, User Service, Discovery Service và API Gateway đều có `register-with-eureka: true`. Điều này có nghĩa là các service này sẽ xuất hiện trong Eureka Dashboard khi chạy thành công.

Ngược lại, Eureka Server được cấu hình `register-with-eureka: false` và `fetch-registry: false`. Đây là cấu hình phù hợp vì Eureka Server trong hệ thống đóng vai trò registry độc lập, không cần tự đăng ký vào chính nó và không cần lấy registry từ một Eureka Server khác.

3.6.3. Cơ chế heartbeat và theo dõi trạng thái service

Sau khi đăng ký thành công, service tiếp tục gửi heartbeat định kỳ đến Eureka Server. Heartbeat cho biết instance vẫn còn hoạt động. Nếu Eureka Server không nhận được heartbeat trong một khoảng thời gian nhất định, instance có thể bị xem là không còn khả dụng và bị loại khỏi danh sách service đang hoạt động. Cơ chế này giúp API Gateway tránh định tuyến request đến những instance không còn hoạt động. Eureka Dashboard cũng cho phép quan sát trạng thái đăng ký của các service trong môi trường phát triển.

Bảng 27. Ý nghĩa của heartbeat trong Eureka

Thành phần	Vai trò trong cơ chế heartbeat
Eureka Client	Gửi heartbeat định kỳ sau khi đăng ký
Eureka Server	Nhận heartbeat và cập nhật trạng thái instance
API Gateway	Lấy registry từ Eureka để định tuyến đến instance khả dụng
Eureka Dashboard	Hiển thị danh sách service và trạng thái đăng ký

Heartbeat không thay thế cho circuit breaker tại Gateway. Eureka giúp Gateway biết service nào đang được đăng ký và còn hoạt động ở mức registry. Trong

khi đó, circuit breaker xử lý trường hợp service vẫn tồn tại trong registry nhưng phản hồi lỗi hoặc phản hồi quá chậm.

3.6.4. Lợi ích của Eureka khi mở rộng hệ thống

Eureka giúp hệ thống đáp ứng tốt hơn yêu cầu mở rộng của kiến trúc microservice. Khi một service được triển khai thêm instance mới, instance đó chỉ cần đăng ký vào Eureka. API Gateway có thể phát hiện instance mới thông qua registry mà không cần thay đổi cấu hình route theo từng địa chỉ cụ thể.

Bảng 28. Lợi ích của Eureka trong Flash Ticket

Lợi ích	Ý nghĩa đối với hệ thống
Tách Gateway khỏi địa chỉ service cụ thể	Gateway gọi service bằng logical service name thay vì IP hoặc port cố định
Hỗ trợ mở rộng ngang	Có thể chạy nhiều instance của cùng một service và đăng ký vào Eureka
Theo dõi service đang hoạt động	Eureka Dashboard hiển thị các service đã đăng ký
Phù hợp với Docker Compose	Các service có thể giao tiếp qua tên container và đăng ký với Eureka Server
Kết hợp được với circuit breaker	Eureka xử lý phát hiện service, còn Resilience4j xử lý lỗi khi service phản hồi chậm hoặc lỗi

Eureka đặc biệt hữu ích với API Gateway vì Gateway là điểm vào tập trung của toàn bộ backend. Nếu không có service discovery, Gateway phải cấu hình cứng địa chỉ của từng service, làm giảm tính linh hoạt khi triển khai. Với Eureka, Gateway chỉ cần biết logical service name, còn danh sách instance cụ thể được Eureka quản lý.

3.7. Thiết kế quản lý cấu hình tập trung bằng Spring Cloud Config Server

Trong hệ thống Flash Ticket, cấu hình của API Gateway và các microservice không được đặt toàn bộ trực tiếp trong từng service. Thay vào đó, hệ thống sử dụng Spring Cloud Config Server để quản lý cấu hình tập trung. Cách thiết kế này giúp các service có thể lấy cấu hình theo tên ứng dụng và theo môi trường chạy.

Config Server đóng vai trò là nơi cung cấp cấu hình tập trung cho các service trong hệ thống. Khi một service khởi động, service đó gửi request đến Config Server để lấy cấu hình tương ứng với tên service và profile đang chạy.

Bảng 29. Vai trò của Config Server trong Flash Ticket

Vai trò	Mô tả
Quản lý cấu hình tập trung	Lưu cấu hình của Gateway và các microservice tại một nơi
Tách cấu hình khỏi mã nguồn service	Service không cần chứa toàn bộ cấu hình chi tiết trong module riêng
Hỗ trợ cấu hình theo môi trường	Có cấu hình mặc định và cấu hình prod cho từng service
Giảm lặp cấu hình	Các nhóm cấu hình như Eureka, RabbitMQ, Redis, Keycloak được quản lý thống nhất
Hỗ trợ triển khai Docker	Khi chạy bằng Docker Compose, service lấy cấu hình từ <code>http://config-server:8888</code>

3.8. Thiết kế event-driven workflow bằng RabbitMQ

Trong hệ thống, RabbitMQ được sử dụng để triển khai các luồng xử lý bất đồng bộ giữa các thành phần trong hệ thống. Các nghiệp vụ như phát hành vé sau thanh toán, gửi email xác nhận, đồng bộ sự kiện sang Discovery Service và đồng bộ người dùng từ Keycloak sang User Service không được xử lý hoàn toàn bằng RESTful API đồng bộ.

Cách thiết kế event-driven giúp tách các bước xử lý theo sự kiện nghiệp vụ. Khi một hành động quan trọng xảy ra, service phát ra event vào RabbitMQ. Service hoặc listener phù hợp sẽ nhận event và xử lý phần việc của mình. Điều này giúp giảm phụ thuộc trực tiếp giữa các service, đồng thời hỗ trợ xử lý lỗi bằng ACK, NACK và Dead Letter Queue.

3.8.1. Vai trò của RabbitMQ trong hệ thống

RabbitMQ đóng vai trò là message broker cho các luồng bất đồng bộ. Thay vì service gọi trực tiếp service khác trong mọi tình huống, hệ thống có thể publish message vào exchange, sau đó RabbitMQ định tuyến message đến queue phù hợp.

Bảng 30. Vai trò của RabbitMQ trong hệ thống Flash Ticket

Vai trò	Mô tả
Tách bước xử lý nghiệp vụ	Tách luồng thanh toán, phát hành vé, tạo QR và gửi email thành các bước riêng
Đồng bộ dữ liệu giữa service	Core Service gửi dữ liệu sự kiện sang Discovery Service để cập nhật embedding
Đồng bộ dữ liệu định danh	Keycloak gửi event sang User Service để đồng bộ người dùng vào MongoDB
Hỗ trợ xử lý lỗi	Message lỗi có thể bị NACK và chuyển vào Dead Letter Queue
Tránh phụ thuộc đồng bộ	Email hoặc Discovery Service lỗi không làm rollback trực tiếp luồng thanh toán đã hoàn tất
Hỗ trợ xử lý tuần tự	Luồng vé được thiết kế theo chuỗi: thanh toán thành công → phát hành vé/QR → gửi email

3.8.2. Luồng xử lý sau khi thanh toán thành công

Luồng thanh toán bắt đầu khi VNPay gửi IPN callback về Core Service. Đây là luồng quan trọng vì liên quan trực tiếp đến trạng thái giao dịch, đơn hàng và vé của người mua.

Trong hệ thống hiện tại, Core Service không chỉ publish event sau thanh toán mà còn thực hiện một số bước quan trọng ngay trong luồng xử lý IPN. Cụ thể, VNPayIPNService kiểm tra Redis idempotency lock, tìm transaction bằng lock DB, kiểm tra trạng thái transaction, xác thực chữ ký VNPay, kiểm tra số tiền, cập nhật transaction, xác nhận order và gọi TicketIssuanceService.issueTickets(orderId). Sau khi các thay đổi dữ liệu được thực hiện, Core Service publish PaymentSuccessEvent

dưới dạng local application event. Event này chỉ được gửi lên RabbitMQ sau khi transaction DB commit thành công thông qua `@TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)`.

Bảng 31. Các bước xử lý sau khi VNPay IPN thành công

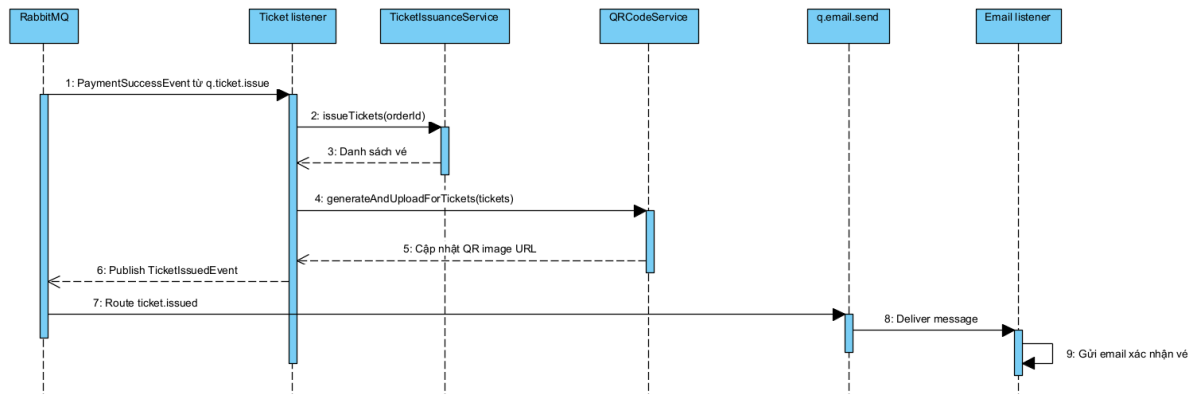
Bước	Thành phần xử lý	Nội dung
1	Core Service	Nhận IPN từ VNPay
2	Core Service	Dùng Redis key <code>vnpay:ipn:{txnRef}</code> để chống xử lý trùng
3	Core Service	Tìm transaction và kiểm tra transaction còn PENDING
4	Core Service	Xác thực chữ ký và số tiền từ VNPay
5	Core Service	Cập nhật transaction và order
6	Core Service	Gọi <code>TicketIssuanceService.issueTickets(orderId)</code>
7	Core Service	Publish local <code>PaymentSuccessEvent</code>
8	Transaction listener	Sau khi DB commit, gửi event vào <code>ex.payment</code> với routing key <code>payment.success</code>

Điểm cần lưu ý là `PaymentSuccessEvent` được publish sau khi commit DB, không publish trực tiếp trong transaction. Cách này tránh tình huống message đã được gửi sang RabbitMQ nhưng dữ liệu trong database chưa commit hoặc bị rollback.

3.8.3. Luồng phát hành vé, tạo QR và gửi email xác nhận

Sau khi `PaymentSuccessEvent` được gửi vào `ex.payment`, RabbitMQ định tuyến message đến queue `q.ticket.issue`. `TicketMessageListener` trong Core Service lắng nghe queue này. Listener phát hành vé thực hiện các công việc sau: gọi lại `TicketIssuanceService.issueTickets(orderId)` theo cơ chế idempotent, tạo và upload ảnh QR bằng `QRCodeService`, đọc thông tin order, đóng gói `TicketIssuedEvent`, sau đó publish event này vào `ex.ticket` với routing key `ticket.issued`.

Việc `issueTickets(orderId)` được gọi lại trong listener không làm tạo trùng vé vì service phát hành vé có kiểm tra vé đã tồn tại. Nếu đơn hàng đã có vé, service trả về danh sách vé hiện có. Cách này giúp listener có thể xử lý lại message an toàn hơn.



Hình 23. Luồng phát hành vé, tạo QR và gửi email xác nhận

Nếu gửi email lỗi, message bị NACK và đi vào `q.email.send.dlq`. Vé đã được phát hành không bị rollback vì gửi email là bước xử lý sau. Đây là điểm phù hợp với thiết kế event-driven: lỗi ở bước thông báo không làm hủy kết quả thanh toán và phát hành vé.

3.8.4. Luồng đồng bộ sự kiện sang Discovery Service

Discovery Service cần dữ liệu sự kiện để phục vụ tìm kiếm ngữ nghĩa, embedding và chatbot. Tuy nhiên, dữ liệu sự kiện gốc được quản lý trong Core Service. Vì vậy, khi sự kiện thay đổi trạng thái hiển thị hoặc nội dung cần đồng bộ, Core Service phát event sang RabbitMQ.

Trong hệ thống, Core Service sử dụng `EventSyncSpringEvent` và `EventSyncPublisher`. Dữ liệu sự kiện được chuẩn bị thành Map trong transaction để tránh lỗi lazy loading khi listener chạy bất đồng bộ sau commit. Sau khi transaction commit, `EventSyncPublisher` gửi dữ liệu sang exchange `ex.discovery` với routing key `event.sync`.

Discovery Service lắng nghe queue `q.discovery.event.sync` bằng `EventDataListener`. Khi nhận message, nếu trạng thái là `DELETED`, Discovery Service xóa embedding theo `eventId`. Nếu không phải `DELETED`, service gọi `EmbeddingIngestionService.upsertEvent()` để cập nhật embedding.

Bảng 32. Luồng đồng bộ sự kiện sang Discovery Service

Bước	Thành phần	Nội dung
1	Core Service	Sự kiện được publish, cập nhật hoặc xóa
2	Core Service	Tạo EventSyncSpringEvent với dữ liệu đã chuẩn bị sẵn
3	EventSyncPublisher	Sau commit, gửi message vào ex.discovery
4	RabbitMQ	Định tuyến event.sync đến q.discovery.event.sync
5	Discovery Service	Nhận message bằng EventDataListener
6	Discovery Service	Upsert hoặc xóa embedding trong PGVector

Cách đồng bộ này giúp Core Service không phải xử lý embedding trong luồng quản lý sự kiện. Discovery Service có thể tập trung vào việc xây dựng dữ liệu tìm kiếm và RAG mà không làm tăng độ phức tạp của nghiệp vụ sự kiện trong Core Service.

3.8.5. Luồng đồng bộ người dùng từ Keycloak sang User Service

Keycloak là hệ thống định danh chính của Flash Ticket. Tuy nhiên, User Service vẫn cần lưu dữ liệu người dùng và organizer trong MongoDB để phục vụ nghiệp vụ riêng của hệ thống. Vì vậy, hệ thống có module keycloak-rabbitmq-spi để lắng nghe sự kiện trong Keycloak và publish event sang RabbitMQ.

Keycloak SPI xử lý các sự kiện người dùng như đăng ký, cập nhật hồ sơ, cập nhật email và xóa tài khoản. Với admin event, SPI cũng xử lý các thao tác tạo, cập nhật hoặc xóa user. Message được publish vào topic exchange keycloak.events.

User Service khai báo queue q.keycloak.user.sync và binding với routing key, nghĩa là queue có thể nhận các message được gửi vào exchange này. KeycloakEventConsumer lắng nghe queue và xử lý dữ liệu theo eventType.

Bảng 33. Đồng bộ người dùng từ Keycloak sang User Service

Event từ Keycloak	Event type trong consumer	Xử lý tại User Service
Register	USER_REGISTER	Tạo user mới trong MongoDB nếu chưa tồn tại
Update profile	USER_UPDATE_PROFILE	Cập nhật thông tin hồ sơ
Update email	USER_UPDATE_EMAIL	Cập nhật email và trạng thái xác thực email
Delete account	USER_DELETE_ACCOUNT	Soft-delete user trong MongoDB
Admin create user	ADMIN_USER_CREATE	Tạo user nếu chưa tồn tại
Admin update user	ADMIN_USER_UPDATE	Cập nhật thông tin user
Admin delete user	ADMIN_USER_DELETE	Soft-delete user

Consumer trong User Service có xử lý idempotent. Khi nhận event đăng ký nhưng user đã tồn tại theo keycloakId, service bỏ qua để tránh tạo trùng. Khi nhận event update nhưng chưa có user trong MongoDB, service có thể tạo user mới từ dữ liệu event.

3.8.6. Manual ACK, NACK và Dead Letter Queue

Các listener RabbitMQ trong Core Service, Discovery Service và User Service đều sử dụng manual ACK. Điều này có nghĩa là message chỉ được xác nhận sau khi xử lý thành công. Nếu xảy ra lỗi trong quá trình xử lý, listener gọi NACK với `requeue=false`, khi đó RabbitMQ chuyển message vào Dead Letter Queue đã cấu hình.

Bảng 34. Cách xử lý ACK/NACK trong các listener

Listener	Queue lắng nghe	Khi thành công	Khi lỗi
TicketMessageListener	q.ticket.issue	basicAck	basicNack(..., requeue=false)
EmailMessageListener	q.email.send	basicAck	basicNack(..., requeue=false)

EventDataListener	q.discovery.event.sync	basicAck	basicNack(..., requeue=false)
KeycloakEventConsumer	q.keycloak.user.sync	basicAck	basicNack(..., requeue=false)

Manual ACK giúp tránh mất message khi consumer nhận message nhưng lỗi xảy ra giữa chừng. Ví dụ, nếu ticket listener nhận message thanh toán thành công nhưng lỗi khi tạo QR hoặc publish event email, message không được ACK và sẽ được chuyển sang DLQ. Nhờ đó, hệ thống vẫn còn dấu vết để kiểm tra và xử lý lại sau.

DLQ không có nghĩa là hệ thống tự động xử lý lại message vô hạn. Trong hệ thống hiện tại, message lỗi được đưa vào DLQ để lưu lại, tránh mất dữ liệu và hỗ trợ kiểm tra thủ công hoặc cơ chế xử lý bổ sung về sau.

3.8.8. Publish event sau khi transaction commit

Một điểm quan trọng trong thiết kế event-driven là không publish các event quan trọng trước khi transaction database hoàn tất. Core Service sử dụng `@TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)` cho các luồng publish RabbitMQ sau khi dữ liệu đã được commit.

Tại luồng thanh toán, VNPayIPNService publish local PaymentSuccessEvent trong transaction. Sau khi transaction commit thành công, listener bắt đầu bộ mới gửi event này vào RabbitMQ. Trong luồng đồng bộ sự kiện, EventSyncPublisher cũng dùng AFTER_COMMIT để gửi dữ liệu sự kiện sang Discovery Service.

Bảng 35. Các event được publish sau transaction commit

Event	Publisher	RabbitMQ exchange	Routing key	Mục đích
PaymentSuccessEvent	VNPayIPNService.handlePaymentSuccessEvent()	ex.payment	payment.success	Kích hoạt luồng phát hành vé/QR và gửi email

EventSyncSpringEvent	EventSyncPublisher.handleEventSync()	ex.discovery	event.sync	Đồng bộ dữ liệu sự kiện sang Discovery Service
----------------------	--------------------------------------	--------------	------------	--

Cách thiết kế này tránh lỗi “phantom message”, tức là message đã được gửi đi nhưng dữ liệu trong database chưa tồn tại hoặc đã rollback. Với hệ thống đặt vé, điều này đặc biệt quan trọng vì các bước phía sau như tạo QR, gửi email hoặc cập nhật embedding đều phụ thuộc vào dữ liệu đã được lưu ổn định trong database

3.9. Thiết kế Redis trong hệ thống

Trong hệ thống Flash Ticket, Redis được dùng ở hai lớp chính: API Gateway và Core Service. Ở API Gateway, Redis hỗ trợ rate limiting. Ở Core Service, Redis được sử dụng thông qua Redisson để xử lý spam lock, distributed lock, idempotency lock và cache trạng thái ghế.

Redis không phải cơ sở dữ liệu nghiệp vụ chính của hệ thống. Dữ liệu nghiệp vụ vẫn được lưu trong PostgreSQL hoặc MongoDB. Vai trò của Redis là hỗ trợ các thao tác cần tốc độ cao, tính nguyên tử và giảm rủi ro xử lý trùng trong các luồng nhạy cảm như đặt vé và thanh toán.

3.9.1. Vai trò của Redis và Redisson

Core Service sử dụng Redisson Client để làm việc với Redis theo dạng các cấu trúc đối tượng như RBucket, RLock và RMapCache. Hệ thống không gọi trực tiếp lệnh Redis thô trong mã nghiệp vụ, mà dùng API của Redisson.

Bảng 36. Vai trò của Redis và Redisson trong hệ thống

Thành phần	Vai trò
Redis	Lưu dữ liệu tạm thời phục vụ rate limiting, lock, chống xử lý trùng và cache trạng thái ghế
Redisson	Client Java giúp Core Service dùng Redis thông qua RBucket, RLock, RMapCache

RBucket.setIfAbsent()	Tạo khóa tạm thời theo cơ chế chỉ set khi key chưa tồn tại
RLock.tryLock()	Lấy distributed lock có thời gian chờ và thời gian tự hết hạn
RMapCache	Lưu map trạng thái ghế theo sự kiện, có TTL

3.9.2. Redis rate limiting tại API Gateway

API Gateway sử dụng RedisRateLimiter của Spring Cloud Gateway. Cấu hình trong hệ thống là:

Bảng 37. Cấu hình rate limiting tại API Gateway

Tham số	Giá trị	Ý nghĩa
Replenish rate	10	Số token được nạp lại mỗi giây
Burst capacity	20	Số token tối đa có thể tích lũy
Requested tokens	1	Số token tiêu thụ cho mỗi request

Gateway sử dụng KeyResolver dựa trên host name của request gửi đến. Như vậy, giới hạn request được áp dụng theo từng nguồn gửi request. Trong cấu hình route hiện tại, requestRateLimiter được gắn cho route core-service. Điều này có nghĩa là các API đi vào Core Service như sự kiện, booking, payment, order, ticket, layout và seat map sẽ chịu giới hạn tần suất ở Gateway.

3.9.3. Redis spam lock khi tạo booking

Khi người dùng tạo booking, Core Service tạo một Redis spam lock trước khi xử lý nghiệp vụ chính. Mục tiêu là chặn tình huống người dùng double-click hoặc gửi nhiều request đặt vé gần như đồng thời.

Bảng 38. Redis spam lock khi tạo booking

Thuộc tính	Giá trị
Key pattern	booking:spam:{userId}:{eventId}
API Redisson	RBucket.setIfAbsent()
TTL	10 giây

Vị trí xử lý	BookingService.createBooking()
Mục đích	Chặn nhiều request tạo booking cùng lúc từ cùng một user cho cùng một event

Nếu key chưa tồn tại, request đầu tiên được tiếp tục xử lý. Nếu key đã tồn tại, hệ thống trả lỗi yêu cầu người dùng chờ trong giây lát. Sau khi xử lý xong, key được xóa trong finally. TTL 10 giây đóng vai trò dự phòng nếu JVM hoặc service gặp lỗi trước khi xóa key.

3.9.4. Redis distributed lock theo ticket type và seat

Sau spam lock, luồng booking tiếp tục sử dụng distributed lock để bảo vệ tồn vé. Hệ thống tách phần lock vào TicketReservationService.

Bảng 39. Distributed lock trong luồng booking

Loại lock	Key pattern	Mục đích
Ticket type lock	lock:tickettype:{ticketTypeId}	Đảm bảo mỗi loại vé chỉ có một request xử lý phần giảm tồn tại một thời điểm
Seat lock	lock:seat:{eventId}:{seatId}	Đảm bảo một ghế cụ thể không bị nhiều request giữ đồng thời

Đối với request có nhiều ticket type hoặc nhiều ghế, hệ thống lấy lock theo thứ tự cố định: ticket type được sắp xếp theo UUID trước, sau đó seat cũng được sắp xếp theo UUID. Cách này giúp giảm nguy cơ deadlock khi nhiều request cùng đặt nhiều loại vé hoặc nhiều ghế.

3.9.5. Redis spam lock khi tạo URL thanh toán

Khi người dùng bấm thanh toán, Core Service tạo URL thanh toán VNPay. Đây cũng là thao tác có nguy cơ bị double-click, tạo nhiều transaction không cần thiết. Vì vậy, Payment Service dùng Redis spam lock trước khi tạo transaction và payment URL.

Bảng 40. Redis spam lock khi tạo URL thanh toán

Thuộc tính	Giá trị
Key pattern	payment:spam:{userId}:{orderId}
API Redisson	RBucket.setIfAbsent()
TTL	5 giây
Vị trí xử lý	PaymentService.initiatePayment()
Mục đích	Chặn double-click khi tạo payment URL

3.9.6. Redis idempotency lock khi xử lý VNPay IPN

VNPay có thể gửi lại IPN nếu không nhận được response đúng hoặc bị timeout. Vì vậy, Core Service cần cơ chế chống xử lý trùng callback thanh toán.

Bảng 41. Redis idempotency lock trong VNPay IPN

Thuộc tính	Giá trị
Key pattern	vnpay:ipn:{txnRef}
API Redisson	RBucket.setIfAbsent()
TTL	30 giây
Vị trí xử lý	VNPayIPNService.processIPN()
Mục đích	Chặn nhiều thread xử lý cùng một transaction VNPay

Khi IPN đến, service tạo lock theo vnp_TxnRef. Nếu lock đã tồn tại, hệ thống xem đây là IPN trùng trong lúc request trước đang xử lý và trả response thành công để VNPay không tiếp tục retry vô ích. Nếu lấy được lock, service tiếp tục kiểm tra transaction, chữ ký, số tiền, trạng thái order và cập nhật dữ liệu.

Redis lock chỉ là lớp bảo vệ đầu tiên. Hệ thống còn dùng lock ở database khi tìm transaction và order, đồng thời kiểm tra transaction còn PENDING hay không trước khi xác nhận thanh toán.

3.9.7. Redis cache trạng thái ghế trong seat map

Đối với sự kiện có sơ đồ ghế, trạng thái ghế cần được đọc thường xuyên ở giao diện người mua. Nếu mỗi lần mở seat map đều truy vấn toàn bộ trạng thái ghế từ database, hệ thống sẽ chịu tải lớn khi sự kiện có nhiều người truy cập cùng lúc. Core Service sử dụng RMapCache để cache trạng thái ghế theo event.

Bảng 42. Redis cache trạng thái ghế

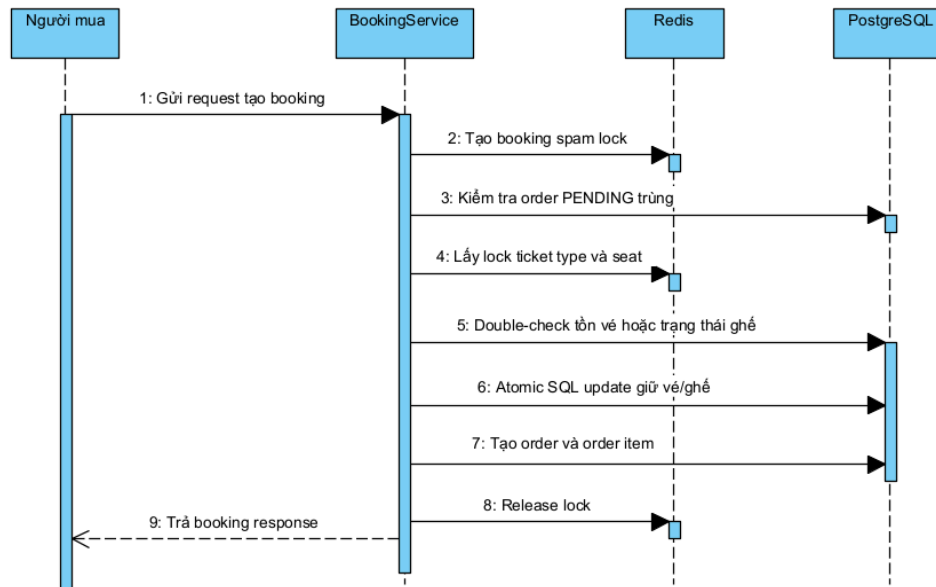
Thuộc tính	Giá trị
Key pattern	seat_status:{eventId}
Kiểu dữ liệu	RMapCache<UUID, String>
Nội dung	Map từ seatId sang trạng thái ghế
TTL	48 giờ
Vị trí xử lý	SeatMapSyncService, SeatStatusCacheListener, SeatBookingService

Khi public seat map được đọc, hệ thống ưu tiên lấy trạng thái ghế từ cache. Nếu cache rỗng hoặc chưa đủ dữ liệu, service đọc từ database rồi làm nóng lại cache. Khi trạng thái ghế thay đổi do reserve, restore hoặc confirm sold, hệ thống phát SeatStatusChangedEvent. Sau khi transaction commit, SeatStatusCacheListener cập nhật cache tương ứng.

Điểm cần lưu ý là ghế không được giữ chỉ vì người dùng bấm chọn trên giao diện. Trong hệ thống, ghế được chuyển sang trạng thái reserved khi người dùng tạo booking thành công.

3.9.8. Kết hợp Redis lock, double-check stock và atomic SQL update

Trong luồng booking, Redis không phải lớp bảo vệ duy nhất. Hệ thống kết hợp nhiều lớp để tránh oversell.



Hình 24. Kết hợp Redis lock và database trong luồng booking

Đối với vé theo số lượng, hệ thống giảm quantityAvailable và tăng quantityReserved bằng câu update có điều kiện. Nếu số row bị ảnh hưởng là 0, nghĩa là tồn vé không đủ và booking bị từ chối.

Đối với vé theo ghế, hệ thống dùng lock theo từng ghế, kiểm tra ghế còn AVAILABLE, sau đó gọi reserveSeatIfAvailable. Câu update chỉ thành công nếu ghế thuộc đúng event, đúng ticket type, đang active và còn available.

Nhờ kết hợp Redis và database như trên, Hệ thống Flash Ticket không phụ thuộc vào một cơ chế duy nhất. Redis giúp giảm cạnh tranh và chống xử lý trùng ở tốc độ cao, còn PostgreSQL vẫn là lớp kiểm soát cuối cùng đảm bảo tính đúng đắn của dữ liệu đặt vé

3.10. Thiết kế luồng nghiệp vụ trọng tâm

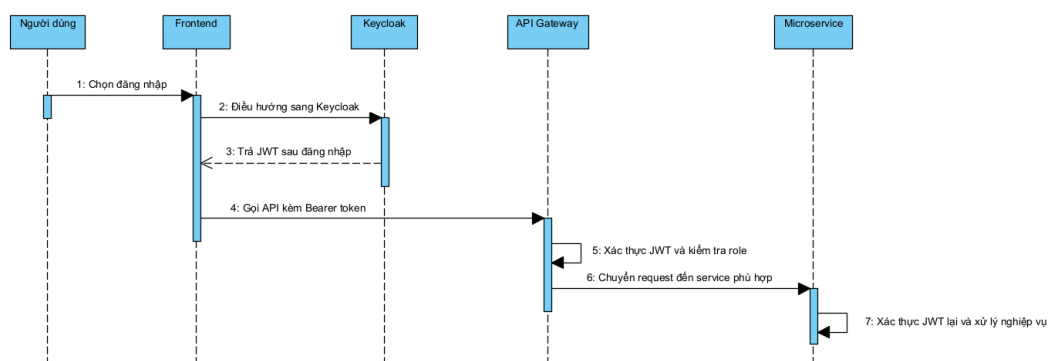
Các luồng nghiệp vụ trọng tâm của Flash Ticket xoay quanh ba nhóm chính: xác thực và phân quyền người dùng, quản lý sự kiện của nhà tổ chức, và quy trình đặt vé từ chọn vé đến thanh toán, phát hành vé, check-in hoặc hoàn vé khi đơn hết hạn. Các luồng này được triển khai kết hợp giữa RESTful API, Keycloak, Redis, PostgreSQL và RabbitMQ.

3.10.1. Luồng đăng nhập và phân quyền

Hệ thống Flash Ticket sử dụng Keycloak làm hệ thống xác thực tập trung. Frontend tích hợp keycloak-js thông qua ReactKeycloakProvider. Khi người dùng đăng nhập thành công, Keycloak cấp access token dạng JWT. Token này được frontend gửi kèm trong header Authorization: Bearer <token> khi gọi API.

API Gateway là lớp kiểm tra đầu tiên. Gateway xác thực JWT bằng cấu hình OAuth2 Resource Server, sau đó trích xuất role từ realm_access.roles. Các role nghiệp vụ được sử dụng gồm BUYER, ORGANIZER và ADMIN, sau đó được ánh xạ thành ROLE_BUYER, ROLE_ORGANIZER, ROLE_ADMIN.

Sau Gateway, các service như Core Service, User Service và Discovery Service cũng được cấu hình Resource Server để xác thực JWT lại ở tầng service. Đây là cơ chế defense in depth, giúp service không phụ thuộc hoàn toàn vào Gateway.



Hình 25. Luồng đăng nhập và phân quyền bằng Keycloak, OAuth2 và JWT

3.10.2. Luồng tạo và quản lý sự kiện

Nhà tổ chức sử dụng nhóm API api/organizer/events để quản lý sự kiện. Nhóm API này yêu cầu role ORGANIZER. Khi tạo sự kiện mới, Core Service lưu sự kiện với trạng thái ban đầu là DRAFT. Organizer có thể cập nhật thông tin sự kiện, cấu hình loại vé, layout, seat map và hình ảnh trước khi công bố.

Bảng 43. Các thao tác quản lý sự kiện của organizer

Thao tác	Hướng xử lý
Xem danh sách sự kiện của mình	Phân trang, lấy theo organizer hiện tại từ JWT
Tạo sự kiện	Tạo trạng thái DRAFT, sinh slug từ tiêu đề
Cập nhật sự kiện	Chỉ cập nhật field được gửi lên
Công bố sự kiện	Chỉ cho publish từ DRAFT, yêu cầu có ít nhất một ticket type active
Hủy sự kiện	Chuyển trạng thái sang CANCELLED
Xóa sự kiện	Soft delete, bị chặn nếu đã có vé bán

Khi sự kiện đã được publish hoặc sự kiện đang publish được cập nhật, Core Service phát EventSyncSpringEvent. Sau khi transaction commit, event được gửi sang RabbitMQ để Discovery Service cập nhật dữ liệu tìm kiếm và embedding.

3.10.3. Luồng thiết kế sơ đồ ghế

Organizer có thể lấy sơ đồ ghế hiện tại và publish dữ liệu từ editor lên backend. Khi publish seat map, Core Service xử lý layout, sector và seat. Với sector dạng SEATED, ghế active phải được gán ticketTypeId. Hệ thống đồng bộ dữ liệu seat inventory, cập nhật bộ đếm của ticket type dạng assigned seat và cập nhật cache trạng thái ghế nếu sự kiện đang publish.

Bảng 44. Các bước xử lý khi publish seat map

Bước	Nội dung
1	Kiểm tra event thuộc organizer hiện tại
2	Lấy layout hiện có và khóa dữ liệu layout để cập nhật
3	Validate sector, seat, ticket type và trạng thái active
4	Lưu sector và seat vào database

5	Đồng bộ event_seat_inventory cho các sector dạng seated
6	Đồng bộ lại số lượng của ticket type dạng assigned seat
7	Cập nhật hoặc làm mới cache seat_status: {eventId}

3.10.4. Luồng đặt vé và giữ vé

Người mua tạo booking qua API api/bookings. User ID được lấy từ subject của JWT. Booking Service kiểm tra sự kiện, loại vé, số lượng mua, trạng thái mở bán và tồn vé trước khi tạo đơn hàng. Luồng booking sử dụng nhiều lớp bảo vệ để tránh bán vượt tồn kho: Redis spam lock, kiểm tra order PENDING trùng, distributed lock theo ticket type/seat, double-check stock và atomic SQL update.

Đối với vé số lượng, hệ thống cập nhật quantityAvailable và quantityReserved. Đối với vé theo ghế, hệ thống tạo bản ghi OrderItemSeat và chuyển trạng thái ghế sang RESERVED. Ghế chỉ được giữ khi booking được tạo thành công, không phải khi người dùng chỉ bấm chọn ghế trên giao diện.

3.10.5. Luồng thanh toán qua VNPay

Sau khi booking được tạo, frontend dùng orderId để gọi api/payments/initiate. Payment Service kiểm tra order thuộc về user hiện tại, order còn hợp lệ, chưa hết hạn và còn ở trạng thái có thể thanh toán. Sau đó service tạo transaction trạng thái PENDING, cập nhật payment method cho order và tạo URL thanh toán VNPay.

Bảng 45. Các bước khởi tạo thanh toán VNPay

Bước	Nội dung
1	Frontend gửi orderId đến POST /api/payments/initiate
2	Payment Service tạo Redis spam lock payment:spam:{userId}:{orderId}
3	Kiểm tra order thuộc về user và còn thanh toán được
4	Tạo transaction PENDING
5	Tạo payment URL đã ký theo cấu hình VNPay
6	Frontend redirect người dùng sang trang thanh toán VNPay

3.10.6. Luồng xử lý VNPay IPN

VNPay IPN được xử lý tại API `api/payments/vnpay-ipn`. Endpoint này được public vì VNPay gọi server-to-server và không gửi JWT. Tuy nhiên, Core Service vẫn xác thực callback bằng chữ ký và kiểm tra số tiền. Luồng IPN sử dụng Redis idempotency lock `vnpay:ipn:{txnRef}` để chống xử lý trùng. Service tìm transaction bằng lock DB, kiểm tra transaction còn PENDING, xác thực chữ ký HMAC từ VNPay, kiểm tra amount, cập nhật transaction và xác nhận order. Sau khi order được xác nhận, Core Servi gọi `TicketIssuanceService.issueTickets(orderId)` và publish local `PaymentSuccessEvent`. Event này được gửi lên RabbitMQ sau khi transaction commit.

3.10.7. Luồng phát hành vé QR và gửi email

Luồng phát hành vé bắt đầu khi order đã được xác nhận thanh toán. `TicketIssuanceService` tạo vé theo từng order item hoặc từng ghế đã giữ. Mỗi vé có `ticketCode` và `qrCodeData` được ký bằng HMAC-SHA256 theo format:

$$\{ticketCode\}|\{eventId\}|\{ticketTypeId\}|\{hmacSignature\}$$

Sau khi `PaymentSuccessEvent` gửi vào RabbitMQ, `TicketMessageListener` nhận message từ `q.ticket.issue`. Listener gọi `issueTickets()` theo cơ chế idempotent, tạo upload QR, sau đó publish `TicketIssuedEvent` sang `ex.ticket`. `EmailMessageListener` nhận message từ `q.email.send` để gửi email xác nhận vé.

3.10.8. Luồng check-in vé tại sự kiện

Bước	Nội dung
1	Nhân viên quét QR và gửi qrData, location lên API
2	Core Service parse QR theo format `ticketCode`
3	Service tính lại HMAC và so sánh với signature
4	Tìm ticket bằng ticketCode với pessimistic lock
5	Kiểm tra vé chưa được sử dụng và có trạng thái VALID
6	Cập nhật trạng thái vé thành USED, lưu checkedInAt, checkedInBy, checkInLocation

Bảng 46. Các bước xử lý check-in vé

3.10.9. Luồng hoàn vé/ghế khi đơn hàng hết hạn

Khi người dùng tạo booking nhưng không thanh toán trong thời gian quy định, order vẫn ở trạng thái PENDING và phần vé/ghế đang được giữ cần được trả lại. Hệ thống xử lý việc này bằng OrderExpirationService. Scheduler chạy mỗi 60 giây, lấy tối đa 50 order PENDING đã quá hạn. Mỗi order được xử lý trong một transaction riêng thông qua OrderExpirationHelper.expireOne().

Bảng 47. Các bước hoàn vé/ghế khi order hết hạn

Bước	Nội dung
1	Scheduler tìm các order PENDING có expiresAt nhỏ hơn thời điểm hiện tại
2	Đánh dấu order thành EXPIRED nếu vẫn còn PENDING
3	Restore stock ticket type: tăng quantityAvailable, giảm quantityReserved
4	Restore seat đã giữ về trạng thái AVAILABLE
5	Release promotion slot nếu order có dùng mã giảm giá
6	Phát event cập nhật cache trạng thái ghế sau khi transaction commit

3.11. Thiết kế cơ sở dữ liệu

Dữ liệu trong Flash Ticket được thiết kế theo hướng tách theo miền nghiệp vụ. Core Service sử dụng PostgreSQL cho nghiệp vụ sự kiện, đặt vé, thanh toán và khuyến mãi. User Service sử dụng MongoDB cho hồ sơ người dùng và organizer. Discovery Service sử dụng PostgreSQL kết hợp PGVector để lưu embedding phục vụ tìm kiếm ngữ nghĩa và RAG.

Cách thiết kế này phù hợp với mô hình microservice vì mỗi service quản lý nhóm dữ liệu chính của mình. Các dữ liệu cần tham chiếu giữa service không luôn được thiết kế dưới dạng khóa ngoại vật lý, mà nhiều trường được lưu dưới dạng logical reference hoặc snapshot để giảm phụ thuộc chặt giữa các miền dữ liệu.

3.12.1. Cơ sở dữ liệu Core Service

Core Service là service có dữ liệu nghiệp vụ lớn nhất trong hệ thống. Service này sử dụng PostgreSQL và chia dữ liệu thành nhiều schema theo từng nhóm nghiệp vụ.

Bảng 48. Các schema chính trong Core Service

Schema	Nhóm dữ liệu	Bảng/entity tiêu biểu
event_schema	Sự kiện, địa điểm, loại vé, layout, seat map	events, venues, categories, ticket_types, event_layouts, event_sectors, event_seats, event_seat_inventory, event_images
booking_schema	Đơn hàng, vé đã phát hành, ghế đã giữ trong order	orders, order_items, order_item_seats, tickets
payment_schema	Giao dịch thanh toán	transactions
promotion_schema	Mã giảm giá và lượt sử dụng	promotions, promotion_usages

Trong event_schema, bảng trung tâm là events. Mỗi event có thể có venue, nhiều category, nhiều ticket type, nhiều hình ảnh và một layout riêng. ticket_types lưu các loại vé của sự kiện, bao gồm giá, số lượng, trạng thái, chế độ inventory và thông tin liên quan đến sector nếu event có seat map.

Trong booking_schema, bảng orders là đơn hàng chính. Mỗi order có nhiều order_items. Sau khi thanh toán thành công, hệ thống tạo các bản ghi thật trong tickets. Với vé theo ghế, order_item_seats lưu các ghế đã được giữ cho từng order item.

Trong payment_schema, bảng transactions lưu giao dịch thanh toán. Transaction tham chiếu logic đến order thông qua order_id, lưu trạng thái giao dịch, mã giao dịch, provider, số tiền, response code và raw response từ VNPay ở dạng JSONB.

3.12.2. Cơ sở dữ liệu User Service

User Service sử dụng MongoDB để lưu dữ liệu người dùng và nhà tổ chức. Dữ liệu xác thực chính vẫn do Keycloak quản lý, còn User Service lưu dữ liệu hồ sơ và dữ liệu nghiệp vụ bổ sung.

Bảng 49. Các collection chính của User Service

Collection	Vai trò
users	Lưu hồ sơ người dùng, email, trạng thái, role, preferences, địa chỉ và liên kết organizer
organizer_profiles	Lưu hồ sơ nhà tổ chức, branding, contact, business info, bank account, verification và thống kê
user_follows	Lưu quan hệ người dùng follow organizer

Trong collection users, trường id và keycloakId liên kết với user ID từ Keycloak. Collection này cũng lưu danh sách role, trạng thái tài khoản, hồ sơ cá nhân, preferences và organizerProfileId nếu người dùng có hồ sơ organizer.

Collection organizer_profiles lưu thông tin tổ chức như tên organizer, slug, loại organizer, mô tả, logo, banner, thông tin liên hệ, thông tin KYC, tài khoản ngân hàng, trạng thái xác minh và một số thống kê.

Collection user_follows được tách riêng thay vì embed vào user document. Collection này có compound unique index theo followerId và organizerProfileId để tránh một user follow cùng một organizer nhiều lần.

3.12.3. Cơ sở dữ liệu Discovery Service và PGVector

Discovery Service sử dụng PostgreSQL với schema discovery_schema. Service này không lưu dữ liệu gốc của sự kiện như Core Service, mà lưu dữ liệu phục vụ tìm kiếm ngữ nghĩa và chatbot.

Bảng 50. Dữ liệu chính trong Discovery Service

Thành phần	Vai trò
event_embeddings	Lưu vector embedding của dữ liệu sự kiện
Metadata embedding	Lưu eventId, organizerId, status, minPrice để phục vụ lọc và truy xuất
Text segment	Lưu nội dung sự kiện đã được chia đoạn
Vector 768 chiều	Vector được tạo từ Google Gemini embedding model

Khi Core Service publish hoặc cập nhật sự kiện, dữ liệu được gửi qua RabbitMQ sang Discovery Service. Discovery Service nhận message, xóa embedding cũ theo eventId, tạo text mô tả sự kiện, chia đoạn với chunk size 500 và overlap 50, sau đó tạo embedding và lưu vào PGVector.

3.12.4. Quan hệ dữ liệu trong nghiệp vụ đặt vé

Nghệp vụ đặt vé đi qua nhiều bảng thuộc event_schema, booking_schema và payment_schema. Về mặt thiết kế, nhiều trường được lưu dưới dạng snapshot tại thời điểm đặt vé hoặc phát hành vé để đảm bảo dữ liệu lịch sử không bị thay đổi khi thông tin event hoặc ticket type bị chỉnh sửa sau này.

Bảng 51. Vai trò các bảng trong luồng đặt vé

Bảng	Vai trò
events	Sự kiện được bán vé
ticket_types	Loại vé, giá, số lượng tổng, số lượng còn lại, số lượng đang giữ
orders	Đơn hàng chính của người mua
order_items	Từng dòng vé trong đơn hàng
order_item_seats	Ghế được giữ cho order item nếu là vé theo ghế
transactions	Giao dịch thanh toán của order
tickets	Vé thật được phát hành sau khi thanh toán thành công

Khi người mua tạo booking, hệ thống tạo orders ở trạng thái PENDING, tạo order_items, giảm quantity_available và tăng quantity_reserved trong ticket_types. Nếu là vé theo ghế, hệ thống tạo thêm order_item_seats và chuyển trạng thái ghế trong event_seat_inventory sang RESERVED.

Khi thanh toán thành công, order chuyển sang CONFIRMED, transaction chuyển sang SUCCESS, hệ thống tạo các bản ghi tickets, giảm quantity_reserved và tăng số vé đã bán của event. Nếu là vé theo ghế, trạng thái ghế được chuyển sang SOLD.

3.12.5. Dữ liệu seat map, ticket type và seat inventory

Seat map trong Flash Ticket được thiết kế để hỗ trợ cả khu vực đứng và khu vực ngồi. Dữ liệu seat map nằm trong event_schema và liên kết với ticket type.

Bảng 52. Các bảng liên quan đến seat map

Bảng	Vai trò
event_layouts	Layout tổng thể của một event, lưu cấu hình canvas, background và metadata
event_sectors	Khu vực trong layout, ví dụ khu đứng hoặc khu ghế ngồi
event_seats	Từng ghế trong sector, gồm tọa độ, row, seat number, label, ticket type và màu
ticket_types	Loại vé có thể gắn với event hoặc sector, hỗ trợ QUANTITY và ASSIGNED_SEAT
event_seat_inventory	Trạng thái inventory của từng ghế trong sự kiện

event_layouts có quan hệ một-một với event. Một layout có nhiều sector. Một sector có thể có nhiều seat. Với sector dạng SEATED, các ghế active phải được gán ticketTypeId. Khi organizer publish seat map, hệ thống đồng bộ event_seat_inventory cho các ghế active.

Trạng thái ghế trong event_seat_inventory gồm các trạng thái như AVAILABLE, RESERVED, SOLD, BLOCKED. Khi người dùng tạo booking thành công, ghế được chuyển sang RESERVED. Khi thanh toán thành công và vé được phát

hành, ghế chuyển sang SOLD. Nếu order hết hạn hoặc bị hủy khi còn PENDING, ghế được trả về AVAILABLE. Thiết kế này giúp hệ thống xử lý rõ ràng hai loại inventory: vé theo số lượng dùng `ticket_types.quantity_available`, còn vé theo ghế dùng `event_seat_inventory.status` kết hợp với `order_item_seats` và `tickets`

3.13. Triển khai hệ thống trên AWS EC2 và Nginx

Để kiểm thử hệ thống trong môi trường gần với thực tế, nhóm triển khai Flash Ticket trên máy chủ Amazon EC2 thuộc nền tảng AWS. EC2 đóng vai trò là server cloud dùng để chạy các thành phần chính của hệ thống như frontend, backend microservices và các dịch vụ hạ tầng cần thiết. Việc triển khai trên EC2 giúp hệ thống có thể được truy cập từ xa thông qua Internet, thay vì chỉ chạy cục bộ trên máy phát triển.

Trong mô hình triển khai này, các service như API Gateway, Core Service, User Service, Discovery Service, Eureka Server, Config Server cùng các thành phần hỗ trợ được cấu hình để hoạt động trên cùng môi trường cloud. Điều này giúp nhóm kiểm thử được khả năng phối hợp giữa các service, các luồng RESTful API, message queue, xác thực bằng Keycloak, xử lý thanh toán và các nghiệp vụ bất đồng bộ của hệ thống.

Bên cạnh EC2, nhóm sử dụng Nginx làm reverse proxy cho hệ thống. Nginx tiếp nhận request từ phía người dùng, sau đó điều hướng đến frontend hoặc API Gateway tùy theo đường dẫn truy cập. Với cách thiết kế này, người dùng chỉ cần truy cập qua một địa chỉ domain hoặc IP công khai, trong khi các service phía sau vẫn có thể được tổ chức theo các port và container riêng biệt.

Nginx cũng giúp hệ thống dễ quản lý hơn trong quá trình triển khai. Thay vì để người dùng truy cập trực tiếp vào từng service, Nginx đóng vai trò lớp trung gian để định tuyến request, che giấu cấu trúc port nội bộ và hỗ trợ cấu hình SSL khi cần triển khai HTTPS. Điều này phù hợp với kiến trúc microservice vì hệ thống có nhiều service độc lập nhưng vẫn cần một điểm truy cập thống nhất cho người dùng cuối.

Nhìn chung, việc triển khai Flash Ticket trên AWS EC2 kết hợp với Nginx giúp nhóm kiểm thử hệ thống trong môi trường cloud, đánh giá khả năng vận hành thực tế và chuẩn bị nền tảng cho việc mở rộng hệ thống trong tương lai.

3.14. Kết luận chương

Chương 3 đã trình bày thiết kế hệ thống Flash Ticket theo kiến trúc microservice, gồm API Gateway, Core Service, User Service, Discovery Service, Eureka Server và Config Server. Hệ thống kết hợp RESTful API cho giao tiếp đồng bộ, RabbitMQ cho xử lý bất đồng bộ, Redis cho chống xử lý trùng và Keycloak cho xác thực, phân quyền.

Chương này cũng làm rõ các luồng nghiệp vụ chính như quản lý sự kiện, thiết kế sơ đồ ghế, đặt vé, thanh toán VNPay, phát hành vé QR, check-in và đồng bộ dữ liệu sang Discovery Service. Các thiết kế về chịu lỗi, bảo mật, cơ sở dữ liệu và chatbot RAG là nền tảng để triển khai và đánh giá hệ thống ở các chương tiếp theo.

CHƯƠNG 4: XÂY DỰNG VÀ TRIỂN KHAI HỆ THỐNG

4.1. Môi trường phát triển và công nghệ sử dụng

Hệ thống Flash Ticket được xây dựng theo kiến trúc microservice, trong đó backend sử dụng Spring Boot và Spring Cloud, frontend sử dụng React với Vite, còn hạ tầng triển khai cục bộ được tổ chức bằng Docker Compose. Việc lựa chọn các công nghệ này nhằm đáp ứng yêu cầu tách service độc lập, giao tiếp qua RESTful API, xử lý bất đồng bộ bằng message queue và hỗ trợ mở rộng hệ thống trong tương lai.

Bảng 53. Công nghệ sử dụng trong hệ thống Flash Ticket

Nhóm công nghệ	Công nghệ sử dụng	Vai trò trong hệ thống
Backend	Spring Boot, Spring Cloud	Xây dựng các microservice
Frontend	React, TypeScript, Vite	Xây dựng giao diện người dùng
API Gateway	Spring Cloud Gateway	Định tuyến request, xác thực, rate limit
Service Discovery	Eureka Server	Đăng ký và phát hiện service
Config	Spring Cloud Config Server	Quản lý cấu hình tập trung
Database	PostgreSQL, MongoDB	Lưu dữ liệu nghiệp vụ và người dùng
Cache/Lock	Redis, Redisson	Distributed lock, chống xử lý trùng
Message Queue	RabbitMQ	Xử lý bất đồng bộ
Authentication	Keycloak, OAuth2, JWT	Đăng nhập và phân quyền
Payment	VNPay Sandbox	Thanh toán trực tuyến
AI Search	LangChain4j, Gemini, PGVector	Chatbot và tìm kiếm ngữ nghĩa
Media	Cloudinary	Lưu ảnh sự kiện, avatar, QR Code
Deployment	Docker Compose	Khởi chạy hạ tầng cục bộ

4.1.1. Backend

Backend của hệ thống được chia thành nhiều service Spring Boot. Mỗi service có mã nguồn, cấu hình và cổng chạy riêng. Các service chính gồm API Gateway, Config Server, Eureka Server, Core Service, User Service và Discovery Service.

Core Service là service có khối lượng nghiệp vụ lớn nhất, phụ trách sự kiện, loại vé, đặt vé, đơn hàng, thanh toán, phát hành vé và gửi email. User Service phụ trách người dùng, hồ sơ organizer và đồng bộ dữ liệu từ Keycloak. Discovery Service phụ trách chatbot, tìm kiếm ngữ nghĩa và hỗ trợ đặt vé thông qua AI.

4.1.2. Frontend

Frontend được xây dựng bằng React, TypeScript và Vite. Giao diện được chia theo các nhóm trang như trang chủ, tìm kiếm sự kiện, chi tiết sự kiện, chọn vé, thanh toán, đơn hàng, vé của tôi, hồ sơ người dùng và khu vực quản lý dành cho organizer. Frontend gọi backend thông qua API Gateway. Axios interceptor được sử dụng để tự động gắn JWT token vào request sau khi người dùng đăng nhập bằng Keycloak.

4.1.3. Cơ sở dữ liệu và hạ tầng

Hệ thống sử dụng PostgreSQL cho dữ liệu nghiệp vụ đặt vé, MongoDB cho dữ liệu người dùng, Redis cho distributed lock và RabbitMQ cho message queue. Ngoài ra, PostgreSQL còn được cài extension PGVector để phục vụ tìm kiếm ngữ nghĩa trong Discovery Service.

4.1.4. Công cụ triển khai

Docker Compose được sử dụng để khởi chạy các thành phần hạ tầng như PostgreSQL, RabbitMQ, Keycloak, MongoDB, Redis, Kafka và pgAdmin. Các backend service được chạy độc lập bằng Maven. Frontend được chạy bằng Vite dev server.

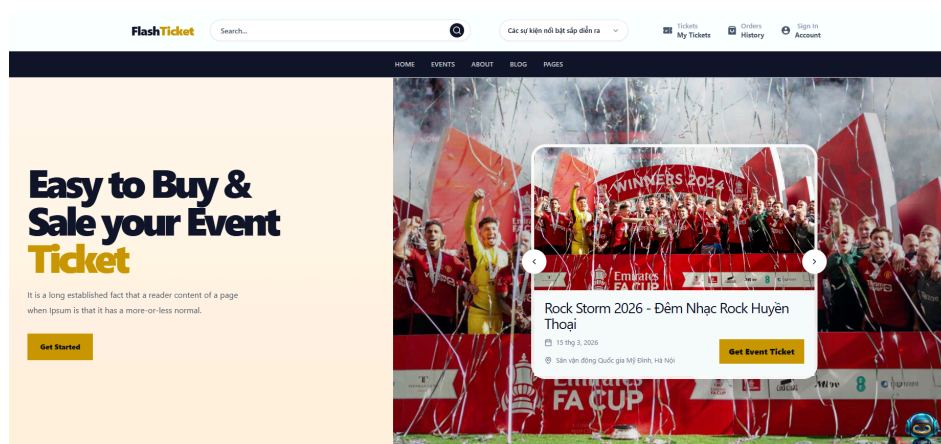
4.3. Giao diện hệ thống

4.3.1. Giao diện trang chủ và tìm kiếm sự kiện

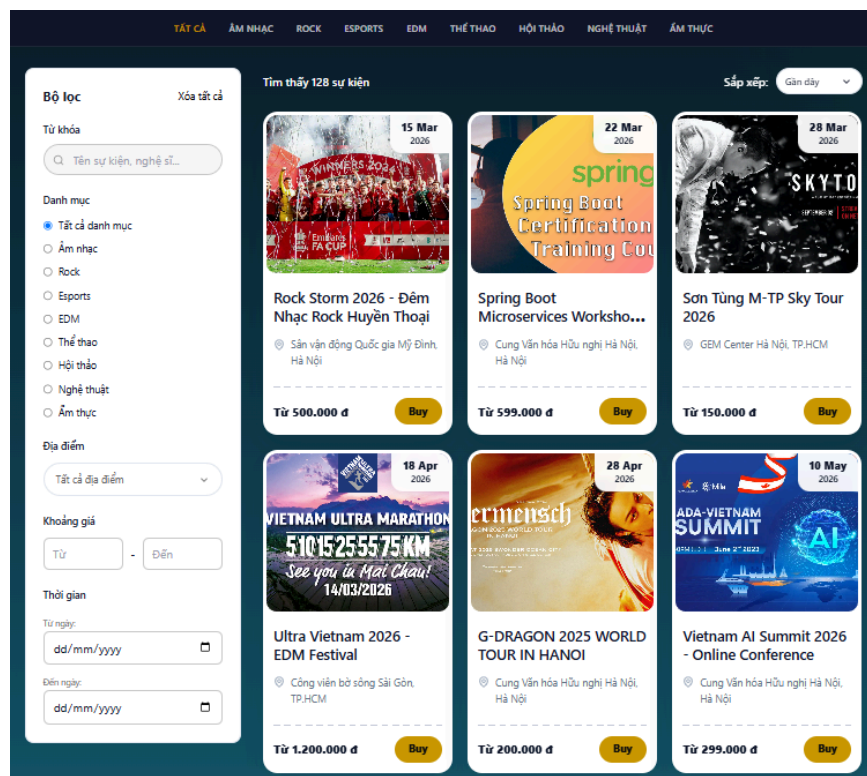
Trang chủ hiển thị danh sách sự kiện nổi bật, danh mục sự kiện và các khu vực nội dung giúp người dùng nhanh chóng tiếp cận sự kiện phù hợp. Trang tìm kiếm cho phép lọc sự kiện theo từ khóa, địa điểm, danh mục hoặc khoảng thời gian.

- Giao diện trang chủ

Hình 26. Giao diện trang chủ



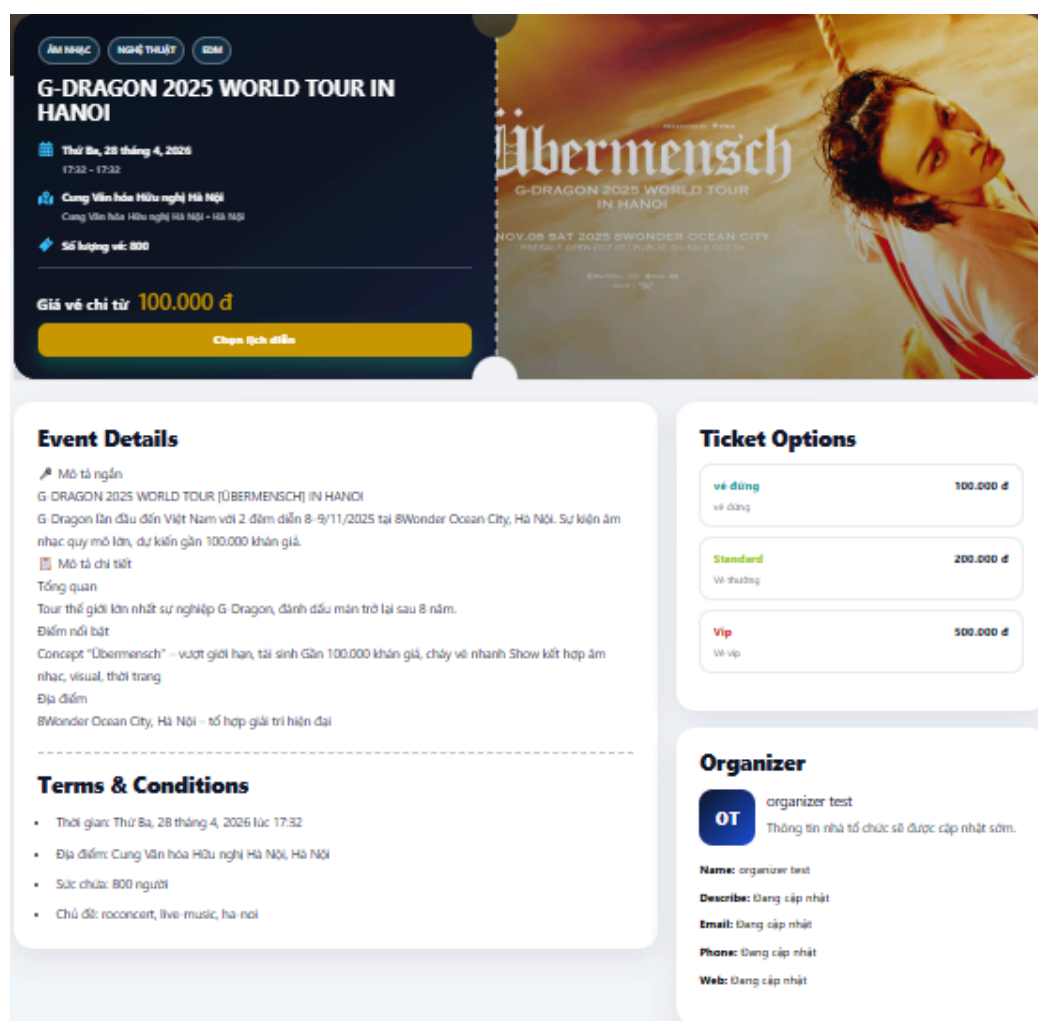
- Giao diện trang tìm kiếm



Hình 27. Giao diện trang tìm kiếm

4.3.2. Giao diện chi tiết sự kiện

Trang chi tiết sự kiện hiển thị thông tin đầy đủ như tên sự kiện, mô tả, hình ảnh, thời gian diễn ra, địa điểm, nhà tổ chức và danh sách loại vé. Từ trang này, người dùng có thể chuyển sang bước chọn vé.



Hình 28. Giao diện trang chi tiết sự kiện

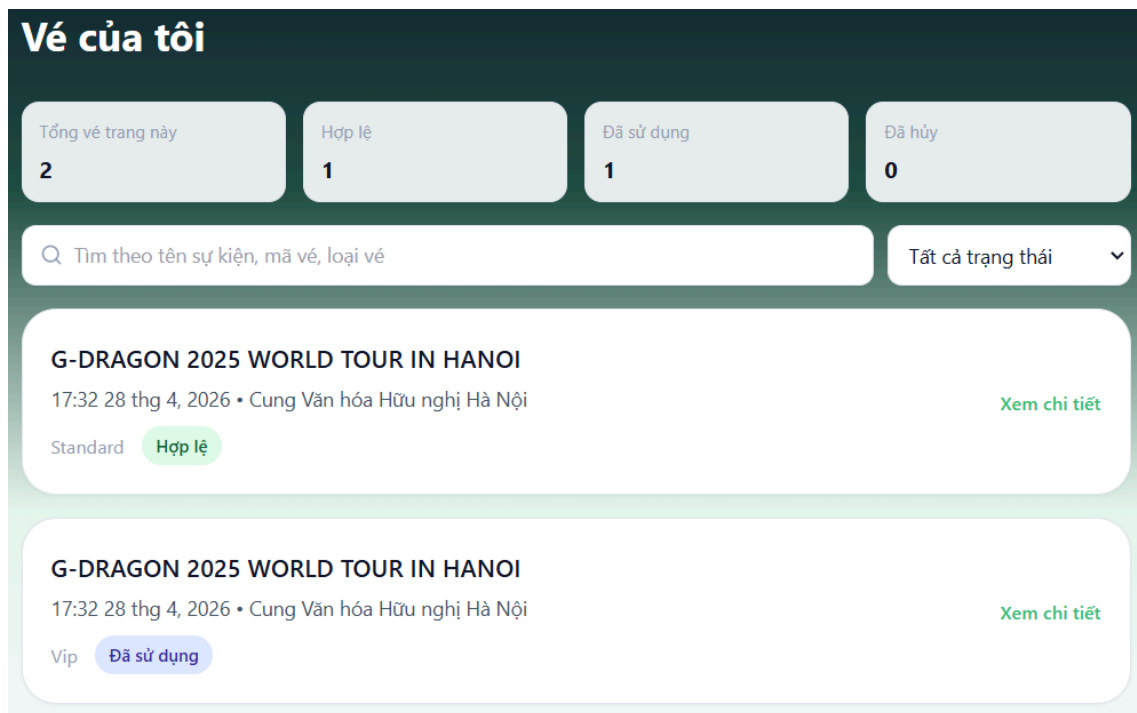
4.3.3. Giao diện chọn vé và thanh toán

Ở bước chọn vé, người dùng chọn loại vé và số lượng vé. Sau đó hệ thống chuyển sang trang checkout để nhập thông tin liên hệ, mã khuyến mãi nếu có và tạo đơn hàng. Khi đơn hàng được tạo, người dùng có thể tiến hành thanh toán qua VNPay.

4.3.4. Giao diện vé của tôi và đơn hàng của tôi

Sau khi thanh toán thành công, người dùng có thể xem danh sách vé đã mua trong trang “Vé của tôi”. Mỗi vé có thông tin sự kiện, loại vé, trạng thái vé và QR Code. Trang “Đơn hàng của tôi” cho phép người dùng theo dõi lịch sử đơn hàng, trạng thái thanh toán và chi tiết từng đơn.

- Giao diện danh sách vé



Hình 29. Giao diện trang danh sách vé

- Giao diện trang chi tiết vé

G-DRAGON 2025 WORLD TOUR IN HANOI



17:32 28 thg 4, 2026 • Cung Văn hóa Hữu nghị Hà Nội

Mã vé	TKT-ORD-20260415-69AC67D6-002
Loại vé	Standard
Người sở hữu	organizer test
Email	organizer@gmail.com
Giá	200.000 đ
Trạng thái	Hợp lệ

Hình 30. Giao diện trang chi tiết vé

- Giao diện trang danh sách đơn hàng

Đơn hàng của tôi

Tổng đơn trang này
4

Chờ thanh toán
0

Đã xác nhận
1

Đã hoàn tiền
0

Tất cả trạng thái

EXO PLANET #6 - EXhOrizon in HO CHI MINH CITY
12:00 14 thg 12, 2026 • ORD-20260518-43B34BE1
Đã hủy

Chi tiết
341.000 đ

G-DRAGON 2025 WORLD TOUR IN HANOI
17:32 28 thg 4, 2026 • ORD-20260415-69AC67D6
Đã xác nhận

Chi tiết
700.000 đ

Hình 31. Giao diện trang danh sách đơn hàng

- Giao diện trang chi tiết đơn hàng

G-DRAGON 2025 WORLD TOUR IN HANOI



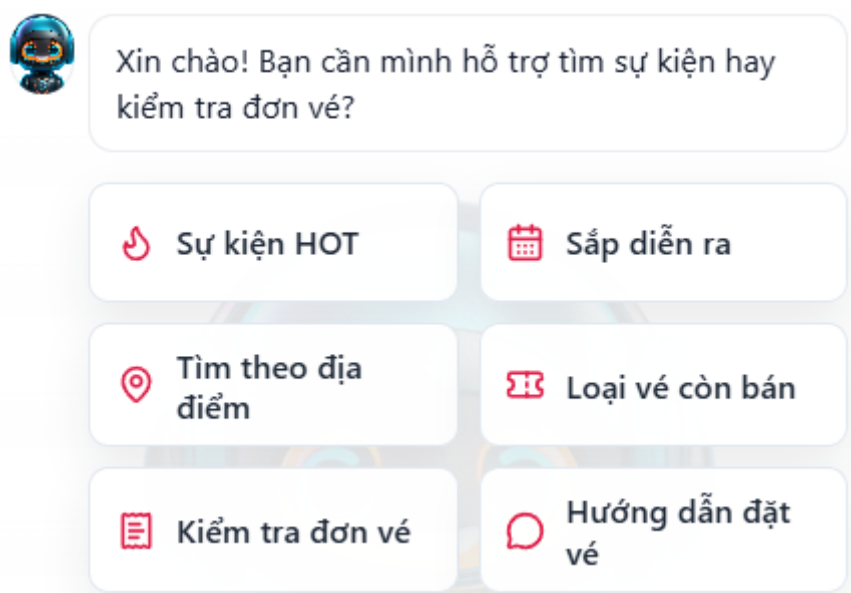
17:32 28 thg 4, 2026 • Cung Văn hóa Hữu nghị Hà Nội

Mã đơn		ORD-20260415-69AC67D6
Trạng thái		Đã xác nhận
Chi tiết vé		
Vip	x1	500.000 đ
Standard	x1	200.000 đ
Tổng cộng		700.000 đ

Hình 32. Giao diện trang chi tiết đơn hàng

4.3.5. Giao diện chatbot hỗ trợ người dùng

Frontend tích hợp chatbot nổi để người dùng có thể hỏi thông tin sự kiện, tìm kiếm sự kiện phù hợp hoặc được hỗ trợ trong quá trình đặt vé. Chatbot gửi request đến Discovery Service thông qua API Gateway.



Hình 33. Giao diện Chatbot

4.4. Kết chương

Chương 4 đã trình bày quá trình xây dựng và triển khai hệ thống Flash Ticket. Nội dung bao gồm môi trường phát triển, công nghệ sử dụng, cách xây dựng hệ thống. Thông qua quá trình xây dựng, hệ thống đã thể hiện được các đặc điểm quan trọng của một ứng dụng microservice: các service được tách biệt theo miền nghiệp vụ, giao tiếp đồng bộ bằng RESTful API, giao tiếp bất đồng bộ bằng RabbitMQ, sử dụng cơ sở dữ liệu phù hợp với từng loại dữ liệu và có cơ chế xác thực, phân quyền, xử lý lỗi cơ bản.

Ở chương tiếp theo, báo cáo sẽ trình bày quá trình thực nghiệm, kiểm thử các luồng nghiệp vụ chính, đánh giá kết quả đạt được, phân tích ưu điểm, hạn chế và hướng phát triển của hệ thống.

CHƯƠNG 5: ĐÁNH GIÁ VÀ KẾT LUẬN

5.1. Ưu điểm của hệ thống

Hệ thống Flash Ticket có ưu điểm lớn nhất là kiến trúc được phân chia rõ ràng theo từng miền nghiệp vụ. Core Service tập trung vào nghiệp vụ đặt vé, User Service quản lý người dùng và organizer, Discovery Service phụ trách chatbot và tìm kiếm thông minh. Việc phân tách này giúp mã nguồn dễ bảo trì hơn so với cách xây dựng nguyên khối.

Hệ thống cũng có luồng nghiệp vụ khá đầy đủ cho một nền tảng đặt vé: người dùng có thể tìm kiếm sự kiện, xem chi tiết, chọn vé, tạo đơn hàng, thanh toán qua VNPay, nhận vé điện tử, nhận QR Code và check-in. Nhà tổ chức có thể quản lý sự kiện, loại vé, hình ảnh và hồ sơ organizer.

Một ưu điểm khác là hệ thống sử dụng kết hợp RESTful API và RabbitMQ. RESTful API phù hợp với các thao tác cần phản hồi trực tiếp, trong khi RabbitMQ giúp xử lý bất đồng bộ các tác vụ như đồng bộ user, phát hành vé, gửi email và đồng bộ dữ liệu sang chatbot.

Về bảo mật, hệ thống tích hợp Keycloak và JWT để xác thực, phân quyền theo role BUYER, ORGANIZER và ADMIN. Gateway và các service đều kiểm tra JWT, giúp tăng mức bảo vệ.

Ngoài ra, việc tích hợp Redis lock trong luồng đặt vé là điểm quan trọng. Đặt vé là nghiệp vụ dễ gặp lỗi cạnh tranh dữ liệu, nên việc dùng spam lock, distributed lock và atomic update giúp giảm nguy cơ oversell.

5.2. Hạn chế còn tồn tại

Khả năng giám sát vận hành còn hạn chế. Hệ thống đã có logging và actuator, nhưng chưa có centralized logging, distributed tracing, dashboard metrics hoặc alerting đầy đủ.

Ngoài ra, một số chức năng quản trị nâng cao chưa hoàn thiện, ví dụ dashboard doanh thu chi tiết, quản lý refund, quản lý toàn bộ đơn hàng, giám sát DLQ và báo cáo vận hành.

5.3. Hướng phát triển trong tương lai

Trước hết, cần bổ sung các chức năng quản trị nâng cao như dashboard doanh thu, thống kê số vé bán ra, tỷ lệ check-in, quản lý refund, quản lý người dùng và theo dõi message lỗi trong RabbitMQ DLQ.

Hệ thống cũng nên được bổ sung observability đầy đủ hơn. Các thành phần như centralized logging, distributed tracing, Prometheus, Grafana và alerting sẽ giúp theo dõi sức khỏe hệ thống tốt hơn khi có nhiều service chạy đồng thời.

Về kiểm thử, cần xây dựng bộ test tự động cho các luồng quan trọng. Đặc biệt, các test về concurrency trong đặt vé, idempotency trong thanh toán và xử lý lỗi message queue là rất cần thiết.

Về AI, Discovery Service có thể được mở rộng để đề xuất sự kiện cá nhân hóa theo lịch sử mua vé, danh mục yêu thích, vị trí địa lý và hành vi tìm kiếm. Chatbot cũng có thể được cải thiện để hỗ trợ nhiều tình huống hơn như kiểm tra đơn hàng, tra cứu vé và gợi ý sự kiện phù hợp.

Cuối cùng, hệ thống có thể được triển khai lên cloud hoặc Kubernetes để thể hiện tốt hơn khả năng mở rộng và triển khai độc lập của microservice.

5.4. Kết luận

Chương 5 nhìn chung, hệ thống đã đáp ứng tốt các yêu cầu chính của một hệ thống microservice. Hệ thống có nhiều service độc lập, sử dụng RESTful API cho giao tiếp đồng bộ, RabbitMQ cho giao tiếp bất đồng bộ, có API Gateway, Service Discovery, xác thực JWT, cơ sở dữ liệu tách biệt và các luồng nghiệp vụ rõ ràng.

Từ quá trình xây dựng và đánh giá, có thể kết luận rằng Flash Ticket System là một hệ thống phù hợp để minh họa việc phát triển ứng dụng đặt vé theo kiến trúc microservice, đồng thời thể hiện được các kỹ thuật quan trọng trong hệ thống phân tán hiện đại.

LINK GITHUB

